Basic Class Creation

Follow along guide to create a class for beginners.

Introduction

This guide is intended to help you create a basic class mod from start to finish.

Goals

Setup for modding⬆️

*Note that the mod I am creating in this example is called Quickster, for your reference.

Lets make a project folder for our mod. In this case, Quickster. All of our mod files and folders will go in here. We will cover it when we get to it, but this folder will essentially be packed by the multitool to be used as our .pak file which we place in the mods folder for your game. We will only need a few folders and files to get a simple class mod running. The bulk of the files we will deal with are .lsx files which are basically xml files. As your mod gets more intricate, you will need to add more folders/files but the following should be all we need to get started making a class. Note the indentation on entries to indicate the file tree structure, ie. Localization folder has the English folder in it and so forth.

Mod folder setup

- Localization : Starting off, we can make a folder called Localization inside your mod folder, in my case my folder called Quickster. This folder/subfolder will deal with your ingame text. For example, we will later include our class name and description, so it appears as so on the character creation screen.
  - English : Should be the language of your ingame text. If you are reading this, I'm guessing yours should also be English.
    - Quickster.loca : The actual file that has the text you will see in the game for your mod. It is not properly readable so when we do start to add any text, we will adding it to the xml file below. We dont need to make this file but once we convert out Quickster.xml file below with the multitool, we will get out Quickster.loca file (Capitalize the first letter when you make it in the multitool)
    - Quickster.xml : The readable version of the localization file for our mod. We will use the multitool to convert this to the .loca file.
- Mods : This will let the multitool know the folder it is housed in is an unpacked mod. When you pack your mod, a new folder will appear here with your mod name and inside it, a meta.lsx. This is very important, and we may need to make some changes to it if your mod isn't working right away when we load up the game.
- Public : The bulk of the work we do will be in this folder and its subfolders.
  - Quickster : This folder name should be the name of your mod. (Capitalize the first letter)
    - ClassDescriptions : Will house your ClassDescriptions.lsx
      - ClassDescription.lsx : Where we will add the base information about our new class
    - Progressions : Will house your class progression information files
      - Progressions.lsx : Where you will create your progression table for your class
    - CharacterCreationPresets : House your ability points presets
      - AbilityDistributionPresets.lsx : Where you set the starting ability points for your class

*If you are unsure about how to make the loca or lsx files, just open a text document and click save as. On the save as menu, change the “Save as type” option to “All types” and change the file name to your mod name. When you convert via the multitool, make sure you specify the file name and extension

Create the most basic class possible⬆️

Creating a class or a mod in general typically involves creating multiple .lsx files (technically it corresponds to another file type I think but lets just say .lsx for simplicity) that are intened to override/modify the games exists version of that file. For example, if we were to unpack Shared.pak (one of the paks in the base game), we would find a whole bunch of files, one of them being ClassDescriptions.lsx. The ClassDescriptions.lsx we made above will need to match the ClassDescription.lsx, in terms of format (not content). Basically, if something exists in the base game that we want to mimic, we need to create the corresponding file and entry for it. Enough background, let’s do it.

*You can either open files individually with some editor or just open the whole folder we created (on the desktop (bg3_mods) in getting started), in visual studio code. If you wanted to match me so its easier to follow along, I did the latter and opened up the folder bg_mods.

ClassDescription.lsx

Lets get started with ClassDescription.lsx since its the best jumping off point imo. We will start by opening up our mod folder in visual studio and navigating to the ClassDescription.lsx file we created. Lets take a look at a entry in the Shared.pak ClassDescription.lsx file for reference on how to start our ClassDescription.lsx:

```
<?xml version="1.0" encoding="UTF-8"?>
<save>
    <version major="4" minor="0" revision="9"
build="314"/>
    <region id="ClassDescriptions">
        <node id="root">
            <children>
                <node id="ClassDescription">
                    <attribute id="BaseHp" type="int32"
value="12"/>
                    <attribute
id="CharacterCreationPose" type="guid" value="0f07ec6e-
4ef0-434e-9a51-1353260ccff8"/>
                    <attribute id="ClassEquipment"
type="FixedString" value="EQP_CC_Barbarian"/>
                    <attribute id="ClassHotbarColumns"
type="int32" value="5"/>
                    <attribute id="CommonHotbarColumns"
type="int32" value="9"/>
                    <attribute id="Description"
type="TranslatedString"
handle="h3aa4b4a4g0076g4635gb363g7891e8f2e495"
```

```
version="10"/>
                    <attribute id="DisplayName"
type="TranslatedString"
handle="h60fa88ecg49b2g402bg9c75g273cbaafd414"
version="1"/>
                    <attribute id="HpPerLevel"
type="int32" value="7"/>
                    <attribute id="ItemsHotbarColumns"
type="int32" value="2"/>
                    <attribute id="LearningStrategy"
type="uint8" value="1"/>
                    <attribute id="Name"
type="FixedString" value="Barbarian"/>
                    <attribute id="PrimaryAbility"
type="uint8" value="1"/>
                    <attribute
id="ProgressionTableUUID" type="guid" value="60bbcc97-
5381-4898-bc15-908c072895de"/>
                    <attribute id="SoundClassType"
type="FixedString" value="Barbarian"/>
                    <attribute id="SpellCastingAbility"
type="uint8" value="6"/>
                    <attribute id="UUID" type="guid"
value="d8cadb42-0ff9-4049-afaf-e5d78d06a399"/>
                    <children>
                        <node id="Tags">
                            <attribute id="Object"
type="guid" value="02913f9a-f696-40cf-acdf-
32032afab32c"/>
                        </node>
                    </children>
                </node>
                <node id="ClassDescription">
                    <attribute
id="CharacterCreationPose" type="guid" value="0f07ec6e-
4ef0-434e-9a51-1353260ccff8"/>
                    <attribute id="Description"
type="TranslatedString"
handle="h5fa8a376gee1cg43e2g8f36g12129d82a292"
version="9"/>
                    <attribute id="DisplayName"
type="TranslatedString"
handle="hc337205dg93e1g4956g9924gb7d641b6e211"
version="1"/>
                    <attribute id="LearningStrategy"
type="uint8" value="1"/>
                    <attribute id="Name"
type="FixedString" value="BerserkerPath"/>
                    <attribute id="ParentGuid"
type="guid" value="d8cadb42-0ff9-4049-afaf-
e5d78d06a399"/>
                    <attribute id="PrimaryAbility"
```

```
type="uint8" value="1"/>
                    <attribute
id="ProgressionTableUUID" type="guid" value="b2a03b63-
809f-4df9-a179-bf5b899082ef"/>
                    <attribute id="SoundClassType"
type="FixedString" value="Barbarian"/>
                    <attribute id="SpellCastingAbility"
type="uint8" value="6"/>
                    <attribute id="UUID" type="guid"
value="32eee7d8-1b2f-4de5-b9ee-78fbd286c6ef"/>
                    <children>
                        <node id="Tags">
                            <attribute id="Object"
type="guid" value="ac3449f7-c09b-4bc0-8634-
fcff97d49279"/>
                        </node>
                    </children>
                </node>
                <node id="ClassDescription">
                    <attribute
id="CharacterCreationPose" type="guid" value="0f07ec6e-
4ef0-434e-9a51-1353260ccff8"/>
                    <attribute id="Description"
type="TranslatedString"
handle="h559c2233gf74fg4fcfg9408gadcb1cba6316"
version="6"/>
                    <attribute id="DisplayName"
type="TranslatedString"
handle="h71cf10b1gadf1g4be5gbe40g466322d942f4"
version="2"/>
                    <attribute id="LearningStrategy"
type="uint8" value="1"/>
                    <attribute id="Name"
type="FixedString" value="TotemWarriorPath"/>
                    <attribute id="ParentGuid"
type="guid" value="d8cadb42-0ff9-4049-afaf-
e5d78d06a399"/>
                    <attribute id="PrimaryAbility"
type="uint8" value="1"/>
                    <attribute
id="ProgressionTableUUID" type="guid" value="48f1760f-
86e7-401c-ad52-633fffbae49e"/>
                    <attribute id="SoundClassType"
type="FixedString" value="Barbarian"/>
                    <attribute id="SpellCastingAbility"
type="uint8" value="6"/>
                    <attribute id="UUID" type="guid"
value="2e585948-d775-451d-b58b-15b75321d11e"/>
                </node>
                ...
            </children>
        </node>
```

```
    </region>
</save>
```

*Note the "..." to indicate more lines after. I have chosen to show just this first class and subclass entry for an example.

As I said before, we are only going to concern ourselves with the most basic stuff to make a class here, starting out at least. As you may have guessed from looking, classes are organized between opening and closing child tags. Each child block houses a class or subclass. In our example we have our base class, Berserker, and right below it is its subclass entry. We can see its a subclass by seeing that the attribute id ParentGuid has a value that matches the UUID of its parent above it. Let's start by copying the general format we see above, essentially only taking the tags of the xml. Lets not worry about subclasses for now:

Quickster\Public\Quickster\ClassDescription\ClassDescription.lsx

```
<?xml version="1.0" encoding="UTF-8"?>
<save>
  <version major="4" minor="3" revision="0"
build="333"/>
    <region id="ClassDescriptions">
        <node id="root">
            <children>
                <node id="ClassDescription">
                </node>
            </children>
        </node>
    </region>
</save>
```

TODO EXPLAIN VERSION INFORMAION. FOR NOW I ADVISE YOU MAKE YOUR VERSION MAJOR SOMETHING HIGH AND BUILD IF YOU ARE RUNNING A STANDALONE MOD. I BELIEVE THIS WILL INTEREFER WITH OTHER MODS THOUGH SO DO WITH CAUTION. WILL UPDATE WHEN MORE KNOWLEDGABLE:

attribute id's

- **BaseHp** : While not required, I see very few cases where you will not want to set a base health for your class. We should put some thought into our class so in my case, the class is going to be a high movement speed melee fighter. He should be zipping in and out of fights without worry but if he does get hit it could be lethal to balance such mobility and damage. I will give him 6 hp for now.

- **CharacterCreationPose** : Since we are not creating any new assets in the class creation portion, don't worry about this for now. It basically is for what pose your character does as the name implies. Let's just yoink an existing one from the Shared ClassDescriptions.lsx, which is typically 0f07ec6e-4ef0-434e-9a51-1353260ccff8.
- **Description** : Description seen when your class is selected on character creation menu. This is technically not required I believe but you will get some generic "Missing Text" message or something where you description should be ingame if you don't include this. We will generate a new handle and place it here. If you went through the getting started (TODO, I GOT LAZY), you already know what to do but for any lazy skippers I will include it once here since im a lazy skipper myself. Hopefully no one skips this part tho… Anyways, lets open up our multitool which has a handy UUID/Handle generator. I would suggest indexing first. In most versions of the tool, we will see something along the lines of "v4 UUID/TranslatedString Handle Generator". In our case, we need a handle, so click the handle checkbox and then click generate. This "hc25377a0g8e44g4645g90c5gee05d6c5e31b" was generated for me. Click it to copy it, then paste it as our value for Description attribute. We have a handle to reference now. I will cover its use later on but click here if you are impatient and want to do things out of order and like to be confused.
- **Name** : Like description, the name of your class that we see on character creation screen. We will generate another handle, like for description. I generated heb6d4970g5238g4bb8ga932g9dd4357d61ed.
- **HpPerLevel** : How much health your class will get when they level up. Please be aware that it takes what value you put here and adds 1 to it. So the 4 I put here plus the basehp i put as 6 will give me 11, not 10.
- **LearningStrategy** : I am unsure if this is required or what it does, sorry. Leave it as 1 for now.
- **Name** : Name, so Quickster for me.
- **PrimaryAbility** : What you scale off of (excluding spell scaling). 1 = dex, 2 = str, 3 = con, 4 = int, 5 = wis, 6 = chr
- **ProgressionTableUUID** : A UUID that references a progression table, which we will make later. For now untick the handle checkbox (last reminder) and generate a UUID. I got b283c957-2267-484a-a6c0-f98479c55e53.
- **SoundClassType** : Might not need, just use another class's SoundClassType for now. I used Paladin.
- **SpellCastingAbility** : Like PrimaryAbility but for your class spell casting modifier.
- **UUID** : A unique UUID for your class. I generated e7b0f304-da32-410e-9e58-6efea9272673.

Here is how my file looks after doing all the above:

Quickster\Public\Quickster\ClassDescription\ClassDescription.lsx

```
<?xml version="1.0" encoding="UTF-8"?>
<save>
```

```
    <version major="4" minor="3" revision="0"
build="333"/>
    <region id="ClassDescriptions">
        <node id="root">
            <children>
                <node id="ClassDescription">
                    <attribute id="BaseHp" type="int32"
value="6"/>
                    <attribute
id="CharacterCreationPose" type="guid" value="0f07ec6e-
4ef0-434e-9a51-1353260ccff8"/>
                    <attribute id="Description"
type="TranslatedString"
handle="hc25377a0g8e44g4645g90c5gee05d6c5e31b"
version="1"/>
                    <attribute id="DisplayName"
type="TranslatedString"
handle="heb6d4970g5238g4bb8ga932g9dd4357d61ed"
version="1"/>
                    <attribute id="HpPerLevel"
type="int32" value="4"/>
                    <attribute id="LearningStrategy"
type="uint8" value="1"/>
                    <attribute id="Name"
type="FixedString" value="Quickster"/>
                    <attribute id="PrimaryAbility"
type="uint8" value="6"/>
                    <attribute
id="ProgressionTableUUID" type="guid" value="b283c957-
2267-484a-a6c0-f98479c55e53"/>
                    <attribute id="SoundClassType"
type="FixedString" value="Paladin"/>
                    <attribute id="SpellCastingAbility"
type="uint8" value="2"/>
                    <attribute id="UUID" type="guid"
value="e7b0f304-da32-410e-9e58-6efea9272673"/>
                </node>
            </children>
        </node>
    </region>
</save>
```

That should be it for now. Remember this is the barebone skeleton of a class just so we can walk around in the world. There are many more ClassDescription attributes we will cover as we make our mod more complex. For now, lets move onto the next file, Progressions.lsx.

Progressions.lsx

Navigate to the Progressions folder and open up Progressions.lsx. Lets look at the Progressions.lsx in the games Shared.pak:

```
<?xml version="1.0" encoding="UTF-8"?>
<save>
    <version major="4" minor="0" revision="9"
build="328"/>
    <region id="Progressions">
        <node id="root">
            <children>
                <node id="Progression">
                    <attribute id="Boosts"
type="LSString"
value="ProficiencyBonus(SavingThrow,Strength);Proficien
cyBonus(SavingThrow,Constitution);Proficiency(LightArmo
r);Proficiency(MediumArmor);Proficiency(Shields);Profic
iency(SimpleWeapons);Proficiency(MartialWeapons);Action
Resource(Rage,2,0)"/>
                    <attribute id="Level" type="uint8"
value="1"/>
                    <attribute id="Name"
type="LSString" value="Barbarian"/>
                    <attribute id="PassivesAdded"
type="LSString"
value="RageUnlock;UnarmouredDefence_Barbarian;Rage_Armo
ur_Message;Rage_NoHeavyArmour_VFX"/>
                    <attribute id="ProgressionType"
type="uint8" value="0"/>
                    <attribute id="Selectors"
type="LSString" value="SelectSkills(233793b3-838a-4d4e-
9d68-1e0a1089aba5,2)"/>
                    <attribute id="TableUUID"
type="guid" value="60bbcc97-5381-4898-bc15-
908c072895de"/>
                    <attribute id="UUID" type="guid"
value="a2198ee9-ea4c-468e-b6b4-22b32d37806e"/>
                </node>
                <node id="Progression">
                    <attribute id="Level" type="uint8"
value="2"/>
                    <attribute id="Name"
type="LSString" value="Barbarian"/>
                    <attribute id="PassivesAdded"
type="LSString" value="DangerSense;RecklessAttack"/>
                    <attribute id="ProgressionType"
type="uint8" value="0"/>
                    <attribute id="TableUUID"
type="guid" value="60bbcc97-5381-4898-bc15-
908c072895de"/>
                    <attribute id="UUID" type="guid"

```

```
value="89986e8a-09b1-4163-b1d0-ddb2332bf3c5"/>
                </node>
                <node id="Progression">
                    <attribute id="Boosts"
type="LSString" value="ActionResource(Rage,1,0)"/>
                    <attribute id="Level" type="uint8"
value="3"/>
                    <attribute id="Name"
type="LSString" value="Barbarian"/>
                    <attribute id="ProgressionType"
type="uint8" value="0"/>
                    <attribute id="TableUUID"
type="guid" value="60bbcc97-5381-4898-bc15-
908c072895de"/>
                    <attribute id="UUID" type="guid"
value="0d4a6b4b-8162-414b-81ef-1838e36e778a"/>
                    <children>
                        <node id="SubClasses">
                            <children>
                                <node id="SubClass">
                                    <attribute
id="Object" type="guid" value="32eee7d8-1b2f-4de5-b9ee-
78fbd286c6ef"/>
                                </node>
                                <node id="SubClass">
                                    <attribute
id="Object" type="guid" value="2e585948-d775-451d-b58b-
15b75321d11e"/>
                                </node>
                            </children>
                        </node>
                    </children>
                </node>
                ...
            </children>
        </node>
    </region>
</save>
```

The Progressions.lsx is a bit more intimidating imo but nothing you cant handle superstar 👍 Progression tables are used to basically give your class whatever things (passives, subclass selection, spell slots, etc) it needs when it levels up, or progresses. While you dont need to add all of these things, your class does require a level 1 progression to make it past the character creation page since its basically a level 1 level up menu. It will make more sense when we get more complex with it. Lets trim the fat again. Here is what we want to start with:

Quickster\Public\Quickster\Progressions\Progressions.lsx

```
<?xml version="1.0" encoding="UTF-8"?>
<save>
    <version major="4" minor="3" revision="0"
build="333"/>
    <region id="Progressions">
        <node id="root">
            <children>
                <node id="Progression">
                </node>
            </children>
        </node>
    </region>
</save>
```

attribute id's

- **Level** : The level for this progression. Another way to say what happens to your class when they reach a certain level. Class creation is basically level, we put 1 here.
- **Name** : The name of your class. I put Quickster.
- **ProgressionType** : There are 3 different progression types the game will look for. Class Progression = 0, 1 = Subclass Progression, and 2 = Race Progression. We are only making a class for now, so I put 0.
- **TableUUID** : Remember the ProgressionTableUUID we generated in our ClassDescriptions.lsx? Here is where we put it. Since progressions happen each level, a progression table encompasses all progressions for all levels. Each unique progression from 1 and onward combine together to make a progression table, ie my level one progression for this class will share the same tableuuid, as will the third, and the fourth, and so on (Subclasses have their own progression tables amongst themselves, not their parent class but I will cover it later). The UUID I generated before was b283c957-2267-484a-a6c0-f98479c55e53 so I will use that.
- **UUID** : A unique UUID for your progression. I generated cfc536d4-10fa-4342-a447-dd952f4a6df7.

Here is how my file looks after doing all the above:

Quickster\Public\Quickster\Progressions\Progressions.lsx

```
<?xml version="1.0" encoding="UTF-8"?>
<save>
    <version major="4" minor="3" revision="0"
build="333"/>
    <region id="Progressions">
        <node id="root">
            <children>
                <node id="Progression">
                    <attribute id="Level" type="uint8"
```

```
value="1"/>
                    <attribute id="Name"
type="LSString" value="Quickster"/>
                    <attribute id="ProgressionType"
type="uint8" value="0"/>
                    <attribute id="TableUUID"
type="guid" value="b283c957-2267-484a-a6c0-
f98479c55e53"/>
                    <attribute id="UUID" type="guid"
value="cfc536d4-10fa-4342-a447-dd952f4a6df7"/>
                </node>
            </children>
        </node>
    </region>
</save>
```

ProgressionDescriptions.lsx

Here is an interesting little file. You should skip this and come back to this later after you get further in the guide (maybe after you finish reading the section on subclasses). We dont technically need this file but I find it valuable since it helps display things our class gets during its progressions/level ups/character creation. I am going to add a quick ProgressionDescription entry to demonstrate what I mean. I will add an entry to show us getting spellslots in our class features on character creation. Lets make the file and add the following:

Quickster\Public\Quickster\Progressions\ProgressionDescriptions.lsx

```
<?xml version="1.0" encoding="utf-8"?>
<save>
  <version major="4" minor="0" revision="4"
build="444"/>
  <region id="ProgressionDescriptions">
    <node id="root">
      <children>
        <node id="ProgressionDescription">
            <attribute id="Description"
type="TranslatedString"
handle="hb2684027ga7c6g49f4gab1eg2473022fb1d4"
version="1"/>
            <attribute id="DisplayName"
type="TranslatedString"
handle="h40ad27e0g3f47g409bg9469g79c71aef7b6g"
version="1"/>
            <attribute id="ProgressionTableId"
type="guid" value="b283c957-2267-484a-a6c0-
f98479c55e53"/>
            <attribute id="Type" type="FixedString"
value="Quickster"/>
            <attribute id="UUID" type="guid"
value="3d63dd44-02d2-4eee-a138-85de94d9a53e"/>
```

```
        </node>
        <node id="ProgressionDescription">
          <attribute id="Description"
type="TranslatedString"
handle="h4cfc6c2bgec3eg4703g93fbgff6640b77db7"
version="1"/>
          <attribute id="DisplayName"
type="TranslatedString"
handle="hcdce7bceg7784g42a0g8445g6a86bfc68fc2"
version="1"/>
          <attribute id="ParamMatch" type="FixedString"
value="0:SpellSlot"/>
          <attribute id="ProgressionTableId"
type="guid" value="b283c957-2267-484a-a6c0-
f98479c55e53"/>
          <attribute id="Type" type="FixedString"
value="ActionResource"/>
          <attribute id="UUID" type="guid"
value="06cd56f0-e623-4fc6-8ca4-1f19cd87c42b"/>
        </node>
      </children>
    </node>
  </region>
</save>
```

Someone more well versed in this could probably explain it better but essentially the first entry is so you can have class features showing for your class. Make sure to include your class name for type. Our second entry is for making our spellslots to put in the class features on the CC page. So we use the same handles and params for the base games progressiondescription but dont forget to make a new uuid for the entry.

This is the standard description required to get spellslots showing on CC. Make a new UUID for each description, they dont get used anywhere except for here. Take your progression table uuid for your class and drop it in both entries for ProgressionTableId. I generated new handles for the first entry but I used the handles from the base game for the second entry. We added this progressiondescription but we dont have the actual progression to add the spellslots on level. Lets go back to our progressions and make the update we need. I came back to add this section again after so my progressions has been fleshed out alot more. For understanding how to get the spell slots to appear in CC, you need to look at the passives attribute and the boosts attibute.

Quickster\Public\Quickster\Progressions\Progressions.lsx

```
...
<node id="Progression">
  <attribute id="Boosts" type="LSString"
value="ActionResource(SpellSlot,2,1);ActionResource(Spe
edForce,2,0);ProficiencyBonus(SavingThrow,Dexterity);Pr
```

```
oficiencyBonus(SavingThrow,Charisma)"/>
  <attribute id="Level" type="uint8" value="1"/>
  <attribute id="Name" type="LSString"
value="Quickster"/>
  <attribute id="PassivesAdded" type="LSString"
value="UnlockedSpellSlotLevel1;Quickster_Zoomies;Quicks
ter_AGoodFit"/>
  <attribute id="ProgressionType" type="uint8"
value="0"/>
  <attribute id="Selectors" type="LSString"
value="SelectPassives(cbbe590a-d630-4fa0-b135-
c16bdf929ad0,1,Quickster_Passives_Level1);SelectSkills(
30dea46e-feae-4637-85ce-
d4cc7a5ae256,2);SelectAbilityBonus(98ef1592-87ad-4b85-
9583-
70c3b12037a9,AbilityBonus,2,1);SelectSpells(6a219531-
09df-4216-9656-58f4661cc632,2,0,,,,AlwaysPrepared)"/>
  <attribute id="TableUUID" type="guid"
value="b283c957-2267-484a-a6c0-f98479c55e53"/>
  <attribute id="UUID" type="guid" value="cfc536d4-
10fa-4342-a447-dd952f4a6df7"/>
  <children>
    <node id="SubClasses">
      <children>
        <node id="SubClass">
          <attribute id="Object" type="guid"
value="6c824863-0bb3-48ac-8a63-0b3915451069" />
        </node>
        <node id="SubClass">
          <attribute id="Object" type="guid"
value="ff3bf3e7-12ee-4183-8ac6-a9aefb0898f4" />
        </node>
      </children>
    </node>
  </children>
</node>
...
```

You can see that I added ActionResource(SpellSlot,2,1) in boosts to give my character the spell slots but equally important I added UnlockedSpellSlotLevel1 which is also important for getting the spellslots to appear in CC. That was really fast but should help you get it set up.

Again, I cover progressiondescriptions.lsx very fast here. More below when I add passives and you can check file insights for attributes. Im only adding this section because like I said above alot of people seem to have trouble getting this to show (myself included) so I wanted to have a quick addition so people can get it going even without fully understanding.

AbilityDistributionPresets.lsx

The last file we need to touch. The AbilityDistributionPresets file is meant to set up your classes starting ability points. Let's grab an entry in the game data that we can copy and paste to our AbilityDistributionPresets.lsx. The entry is pretty self explanatory, just set the ability points how you want for your class. I set mine below. Remember the classUUID we generated before in ClassDescriptions.lsx? Make sure to put that for your ClassUUID here as well as generate another UUID for this AbilityDistributionPreset (less detail on repetitive stuff as we go along since I'm assuming you are following this guide from the beginning).

Quickster\Public\Quickster\CharacterCreationPresets\AbilityDistributionPresets.lsx

```
<?xml version="1.0" encoding="UTF-8"?>
<save>
    <version major="4" minor="3" revision="0"
build="333"/>
    <region id="AbilityDistributionPresets">
        <node id="root">
            <children>
                <node id="AbilityDistributionPreset">
                    <attribute id="Charisma"
type="int32" value="10"/>
                    <attribute id="ClassUUID"
type="guid" value="e7b0f304-da32-410e-9e58-
6efea9272673"/>
                    <attribute id="Constitution"
type="int32" value="12"/>
                    <attribute id="Dexterity"
type="int32" value="15"/>
                    <attribute id="Intelligence"
type="int32" value="14"/>
                    <attribute id="Strength"
type="int32" value="8"/>
                    <attribute id="UUID" type="guid"
value="cfb21b13-3ac5-4c9d-9dea-2657a88747ed"/>
                    <attribute id="Wisdom" type="int32"
value="13"/>
                </node>
            </children>
        </node>
    </region>
</save>
```

Localizations ⬆️

*You will be updating your localization file a lot so I'm going to broadly cover it here once but please understand that anytime you need some sort of display text and you generate a handle, you will need to update your localization file. I won't cover it again beyond this, so if you see something about adding a handle, know you

need to create the appropriate entry in your localization file because I won't specify that I am doing that anymore.

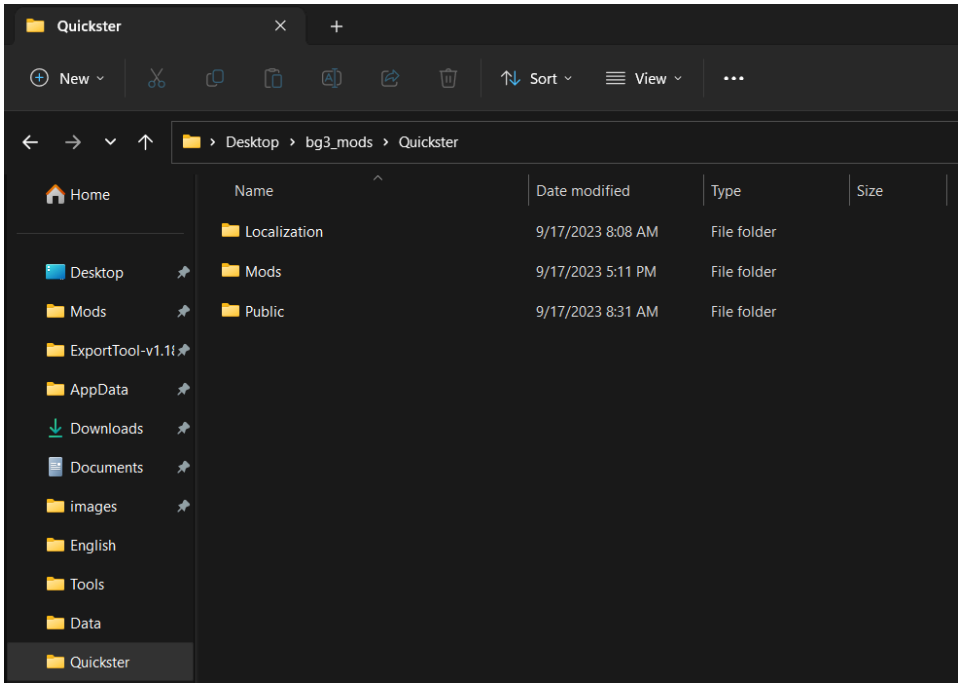
If we were to load up our class right now, we would see a generic skull icon where our class is and some weird text, I think a bunch of ?'s since we haven't added any text to be displayed for our class and its description (and the icon but I will cover that later TODO). Do you remember earlier in our ClassDescriptions.lsx, we generated two handles? We need to put those to use. Inside that Localization/English folder, create your localization file for your mod, so for me Quickster.loca.xml. When you pack your mod, it should convert to a loca file. Let's take a look at my xml file.

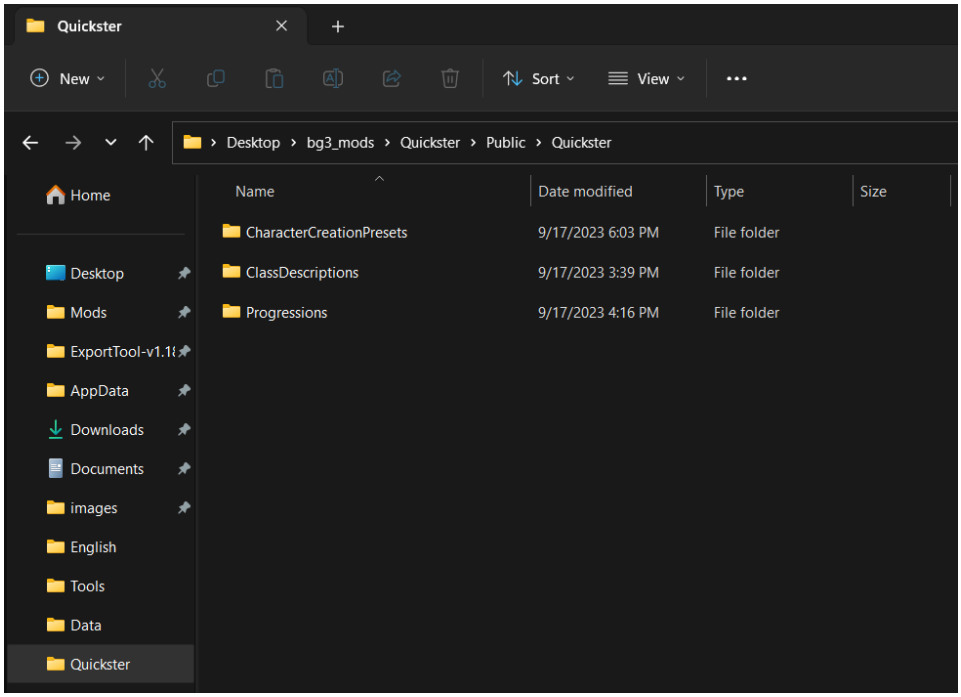
```
<?xml version="1.0" encoding="utf-8"?>
<contentList>
    <content
contentuid="heb6d4970g5238g4bb8ga932g9dd4357d61ed"
version="1">Quickster</content>
    <content
contentuid="hc25377a0g8e44g4645g90c5gee05d6c5e31b"
version="1">Gifted a magical set of boots by a
mysterious god, the quickster attains speeds once
thought impossible.</content>
</contentList>
```

You can see that I took the handles I made for DisplayName and Description in ClassDescriptions.lsx and made them my contentuid in this file. I then write the text I want displayed for that handle. TODO FLESH OUT SECTION ON LOCALIZATIONS

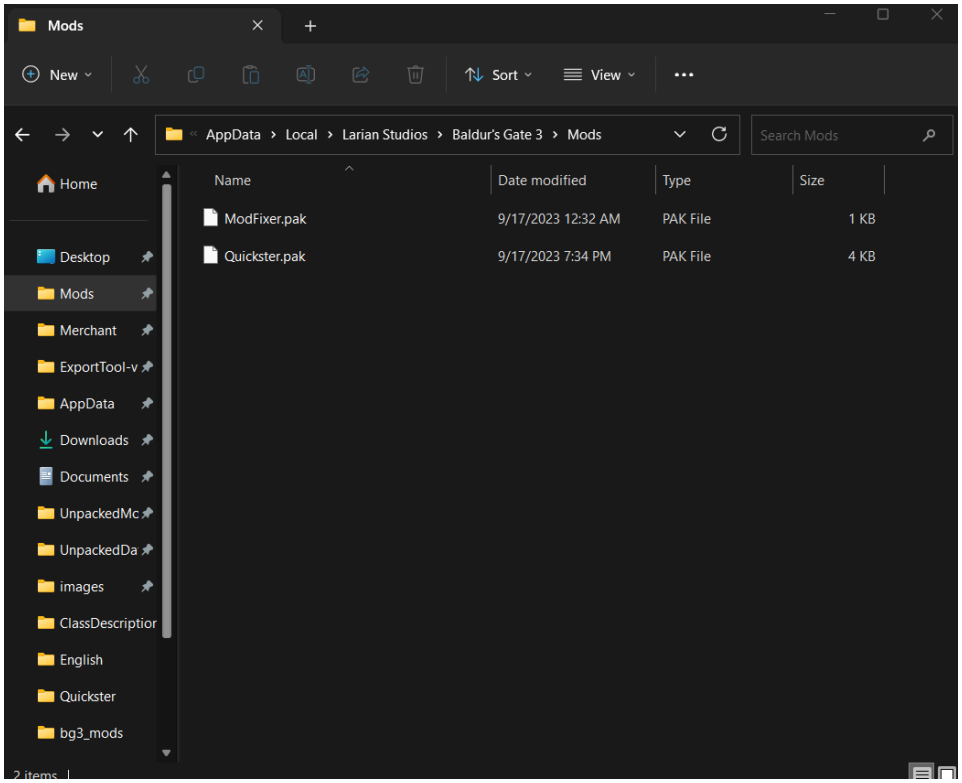
Pack and load your mod⬆️

Almost ready to start running around as your new class. Here is a few images to reference to make sure you are all caught up. First is my project folder for my mod, Quickster. It has all my files and folders for my mod. After that is location of my mod files, which starting from my root folder has the path of Quickster\Public\Quickster.





Anyways, lets pack up our files. I suggest you pin your mod folder(in my case, Quickster) to your quick access. Open up the multitool app. If you haven configured the multitool yet, visit the getting started but basically make sure your packed mods get placed in the mods folder in the required %appdata% subdirectory. Lets drag our folder over to the multitool to start packing. It should ask you to submit an author and description for the mod. It is doing this to create the meta.lsx file. Once done packing, navigate to the mods folder to make sure your mod is there. In my case, Quickster.pak:



*Your Mod Fixer should also be there, like in the image, if you are on a Pre-Patch 7 version of the game. Otherwise, Mod Fixer is not needed anymore.

Lets open up our BGModManager application. If you don't see your mod name at all (it should be on the right hand side if this is the first time you opened the manager since creating your mod pak), hit refresh and it should appear. Drag your mod to the left side and hit ctrl + s to save the mod load order. Alright, its time! Hit ctrl + shift + g to launch the game!

You should see your class in character creation. You may get some message about how "We were unable to create a working story..." when you hit "New Game". This just means Mod Fixer is working correctly and ensuring your story doesn't break when you use mods , just hit esc to cancel the cut scene and hit accept on the message. If you are seeing the following picture below instead of character creation, you may have forgotten to install the Mod Fixer or there is some issue with the Mod Fixer.

The previous info about Mod Fixer is only relevant if you are on a Pre-Patch 7 version of the game. If you are on Patch 7 or 8, you do not need Mod Fixer anymore.

Add class skills/proficiencies and bonus ability points ⬆️

While we have a working character at this point, it is definitely lacking a lot of common options on character creation. You will see that there is no way to spec bonus ability points and that you don't have any skills or proficiencies for the class, among other things. Let's address the more generic things like that and worry about things that would be specific to your class like a passive selection menu on character creation for example, for later. Like before, we will need to create some more folders and files inside out existing mod folder. Make your way to the {mod_name}\Public{mod_name}\ directory so in my case, Quickster\Public\Quickster, and begin making the required folders/files.

- DefaultValues : Make a folder called DefaultValues. This folder will deal with default selection options on character creation and more.
  - Abilities.lsx : Used to set the default selected bonus ability points for your class (the +2 and +1).
  - Skills.lsx : Used to set the default selected skills for your class.
- Lists : Not sure how to describe this folder, it has lists you can reference in other files 🏅 (like spell lists, ability lists, etc)
  - AbilityLists.lsx : Used to set what options you class can select for its bonus ability points

Abilities.lsx

Lets grab an entry from an existing Abilities.lsx entry and modify it for our use. This file is used to determine what abilities the class will get the +2 and +1 bonus when you apply your points. Let me cover the attributes I changed and below you can see an example of the file after.

attribute id's

- **Add** : What you plan on having as the default. I set dex and charisma for my class.
- **Level** : The level for this. We are using this value at level 1 (character creation) so I set at 1.
- **TableUUID** : This refers to a progression table. We need to use the same progression table we have been using. The UUID i generated before was b283c957-2267-484a-a6c0-f98479c55e53 so I will use that.
- **UUID** : A unique UUID for ability default. I generated 370eaae8-7b5b-47a9-b04a-0c2ebdd510cb.

Here is how my file looks after doing all the above:

Quickster\Public\Quickster\DefaultValues\Abilities.lsx

```
<?xml version="1.0" encoding="UTF-8"?>
<save>
    <version major="4" minor="3" revision="0"
build="333"/>
    <region id="DefaultValues">
        <node id="root">
            <children>
                <node id="DefaultValue">
                    <attribute id="Add" type="LSString"
value="Charisma;Intelligence"/>
                    <attribute id="Level" type="int32"
value="1"/>
                    <attribute id="TableUUID"
type="guid" value="98cafd28-0703-426b-9622-
d6c4e128bef1"/>
                    <attribute id="UUID" type="guid"
value="bda7d8b9-2cb6-4691-a0f3-e0e2cf8416d3"/>
                </node>
            </children>
        </node>
    </region>
</save>
```

AbilityLists.lsx

Did you load up your game all excited to see your new preselected options for abilities like I did? Then you were also still met with no menu for bonus ability points. We made a file for the default values for the list of options but not the list of options themselves! Lets grab an entry from an existing AbilityLists.lsx entry and modify it for our use. This file is used to determine what items should make up the list of bonus abilities (the +2 and +1 bonus when you apply your points). Let me cover the attributes I changed and below you can see an example of the file after.

attribute id's

- **Abilities** : We want all abilities to be able to have a bonus point given to them if we want so lets add all the attributes in the game like most classes would have.
- **UUID** : A unique UUID for ability default. I generated 98ef1592-87ad-4b85-9583-70c3b12037a9.

Here is how my file looks after doing all the above:

Quickster\Public\Quickster\Lists\AbilityLists.lsx

```
<?xml version="1.0" encoding="utf-8"?>
<save>
  <version major="4" minor="3" revision="0"
build="333"/>
  <region id="AbilityLists">
    <node id="root">
      <children>
        <node id="AbilityList">
          <attribute id="Abilities" type="LSString"
value="Strength, Dexterity, Constitution, Intelligence,
Wisdom, Charisma" />
          <attribute id="UUID" type="guid"
value="98ef1592-87ad-4b85-9583-70c3b12037a9" />
        </node>
      </children>
    </node>
  </region>
</save>
```

Skills.lsx

Lets grab an entry from an existing Skills.lsx entry and modify it for our use. This file is used to determine what our default selected skills will be (for our class specifically). Like Abiities.lsx, we will need to create a SkillList.lsx entry for the default settings we make here. Let me cover the attributes I changed and below you can see an example of the file after.

attribute id's

- **Add** : What you plan on having as the default. I set dex and charisma for my class.
- **Level** : The level for this. We are using this value at level 1 (character creation) so I set at 1.
- **TableUUID** : This refers to a progression table. We need to use the same progression table we have been using. The UUID i generated before was b283c957-2267-484a-a6c0-f98479c55e53 so I will use that.
- **UUID** : A unique UUID for ability default. I generated 72369441-dd7d-4908-9481-06417ca665bb.

Here is how my file looks after doing all the above:
Quickster\Public\Quickster\DefaultValues\Skills.lsx

```
<?xml version="1.0" encoding="UTF-8"?>
<save>
    <version major="4" minor="3" revision="0"
build="333"/>
    <region id="DefaultValues">
        <node id="root">
            <children>
```

```
                <node id="DefaultValue">
                    <attribute id="Add" type="LSString"
value="Athletics;Acrobatics"/>
                    <attribute id="Level" type="int32"
value="1"/>
                    <attribute id="TableUUID"
type="guid" value="b283c957-2267-484a-a6c0-
f98479c55e53"/>
                    <attribute id="UUID" type="guid"
value="72369441-dd7d-4908-9481-06417ca665bb"/>
                </node>
            </children>
        </node>
    </region>
</save>
```

SkillLists.lsx

Like Abilities.lsx and AbilityLists.lsx, we need to create a SkillLists.lsx for Skills.lsx. This will allow your class to pick certain skills/proficiencies, with the Skills.lsx holding the default ones when you click to open the menu for it.

attribute id's

- **Skills** : Will take a list of skills you want your class to be able to select on skill selection on character creation. My class will be speed based and a bit of magic to maybe give buffs to mobility? Idk, so anyways I added Athletics, Acrobatics, Arcana, SleightOfHand, Perception for values.
- **UUID** : A unique UUID for ability default. I generated 30dea46e-feae-4637-85ce-d4cc7a5ae256.

Here is how my file looks after doing all the above:

Quickster\Public\Quickster\Lists\SkillLists.lsx

```
<?xml version="1.0" encoding="utf-8"?>
<save>
  <version major="4" minor="3" revision="0"
build="333"/>
  <region id="SkillLists">
    <node id="root">
      <children>
        <node id="SkillList">
          <attribute id="Skills" type="LSString"
value="Athletics, Acrobatics, Arcana, SleightOfHand,
Perception"/>
          <attribute id="UUID" type="guid"
value="30dea46e-feae-4637-85ce-d4cc7a5ae256"/>
        </node>
      </children>
    </node>
```

```
  </region>
</save>
```

Linking

We have created our files but we dont have anything that refers to them. Our progressions.lsx determines what sort of stuff we see upon character creation/levelups. Since we need to see this menu for bonus ability points when our character is created, we need this tied to our level one progression. Open up our progressions file we made before. I will cover this more later (TODO link), but we need to add a selector in our progressions table. Below you can see how I did it, but make sure to include your uuid's that you made for AbilityLists.lsx and SkillLists.lsx, not mine.

Quickster\Public\Quickster\Progressions\Progressions.lsx

```
<?xml version="1.0" encoding="utf-8"?>
<save>
  <version major="4" minor="3" revision="0"
build="333"/>
  <region id="Progressions">
    <node id="root">
      <children>
        <node id="Progression">
          <attribute id="Level" type="uint8"
value="1"/>
          <attribute id="Name" type="LSString"
value="Quickster"/>
          <attribute id="ProgressionType" type="uint8"
value="0"/>
          <attribute id="Selectors" type="LSString"
value="SelectSkills(30dea46e-feae-4637-85ce-
d4cc7a5ae256,2);SelectAbilityBonus(98ef1592-87ad-4b85-
9583-70c3b12037a9,AbilityBonus,2,1)"/>
          <attribute id="TableUUID" type="guid"
value="b283c957-2267-484a-a6c0-f98479c55e53"/>
          <attribute id="UUID" type="guid"
value="cfc536d4-10fa-4342-a447-dd952f4a6df7"/>
        </node>
      </children>
    </node>
  </region>
</save>
```

Great, we can load this up and see our class and all its glorious default settings. Right now we have alot of basic stuff that most classes have. All we did was override alot of default settings. So let's actually make something unique about this class. We already used the selector attribute, let's see what else we can do with selectors.

*Up to this point you could have probably copy and pasted alot of this since most classes would require the basic setup I did above. Now things will be more specific to what my class needs. Understand why im doing it and think about how it could be useful for your class.

Tags ⬆️

*This tags section isnt that important for most people. Unless you are getting indepth I wouldnt worry about this right now nor did I flesh this section out for that purpose. This is to just show what can be done, if you want to get more indepth you will need to vist the tags page (TODO ADD TAGS PAGE)

If you scrolled through the ClassDescriptions.lsx (or some other files that have tags), you may have noticed that in the node for ClassDescription, after the attributes, we see another entry, children, which has its own sub nodes. We can see a node id called Tags in there. Tags are pretty important for connecting things in game to whatever you need, in this case a class. Basically using tags fills out your characters character sheet. This is important because the game will open up more options for you if your character is 'tagged' to get it, like say special dialogue for your new class should the tag appear as dialogue option.

Make tag file

We are going to want to start by making a folder called Tags and a .lsx file to go in it. The tags folder will be placed in your \Public\{mod_name}\ directory and the .lsx file should be a new unique uuid, I generated 35add446-b710-4ad1-8dbc-36f99aecc6d5. Lets drop some starter info into our tag file and go over it.

Quickster\Public\Quickster\Tags\35add446-b710-4ad1-8dbc-36f99aecc6d5.lsx

```
<?xml version="1.0" encoding="utf-8"?>
<save>
  <version major="4" minor="3" revision="0"
build="333"/>
  <region id="Tags">
    <node id="Tags">
      <attribute id="Description" type="LSString"
value="|Quickster|"/>
      <attribute id="DisplayDescription"
type="TranslatedString"
handle="hd2b088dcgec45g4e65g9be5g4c6f689e46c6"
version="1"/>
      <attribute id="DisplayName"
type="TranslatedString"
handle="h8cb04d43g3a62g4f3egb4e2g567dea9c7799"
version="2"/>
      <attribute id="Icon" type="FixedString"
value=""/>
      <attribute id="Name" type="FixedString"
value="Quickster_Internal_Name"/>
      <attribute id="UUID" type="guid" value="35add446-
```

```
b710-4ad1-8dbc-36f99aecc6d5"/>
      <children>
        <node id="Categories">
          <children>
            <node id="Category">
              <attribute id="Name" type="LSString"
value="Code"/>
            </node>
            <node id="Category">
              <attribute id="Name" type="LSString"
value="Dialog"/>
            </node>
            <node id="Category">
              <attribute id="Name" type="LSString"
value="Class"/>
            </node>
            <node id="Category">
              <attribute id="Name" type="LSString"
value="CharacterSheet"/>
            </node>
          </children>
        </node>
      </children>
    </node>
  </region>
</save>
```

*Note the UUID used is the same as the one I generated for my file name

I have been trying to keep this section of the tutorial pretty isolated, in that I was mostly sticking to things in the shared.pak. I dont want to dive to much into it here but this next part regarding tags will be using the gustav.pak. If you arent too concerned about integrating your class with the game (getting custom dialog options, etc) you can definetly skip this section and head to selectors . Otherwise, I will only be covering the basics of tag usage here. You can get more indepth here (TODO add tag link).

Use Tag

Like I mentioned above, I want to add something unique for my class via its tag. Lets add a custom dialog option for the Quickster class for the cutscene at the beginning of the game, when the player clicks the brinepool with tadpoles in it. First we will need to locate the file that handles the dialog of that scene. You can find this in the Gustav\Story\Dialogs\Tutorial\TUT_Start_Brinepool.lsj. Its a pretty big file so I dont want to post the whole thing here but lets start by recreating the file path in our project and copying that lsj file over to it. Starting from my main project folder I made the folders so I now have this path Quickster\Mods\Quickster\Story\Dialogs\Tutorial\TUT_Start_Brinepool.lsj. Lets start by moving to the bottom of the file. Typically we can see the initial message down here, as well as the dialog options we get from it. We should see 3 child

nodes that are uuids. We want to add our own custom one in here, for our class. I generated the uuid of 3c243807-5db3-4172-93a0-0dad00d710f5 so i made a child node for it.

Quickster\Mods\Quickster\Story\Dialogs\Tutorial\TUT_Start_Brinepool.lsj

```
...
"Tags" : [ {} ],
"UUID" : {"type" : "FixedString", "value" : "720e1070-
fc2a-4262-85a3-7b288992af64"},
"addressedspeaker" : {"type" : "int32", "value" : -1},
"checkflags" : [ {} ],
"children" : [
   {
      "child" : [
         {
            "UUID" : {"type" : "FixedString", "value" :
"6cdda042-9c82-4e29-b22e-4ef740acc767"}
         },
         {
            "UUID" : {"type" : "FixedString", "value" :
"3f3404b2-43db-42c2-8dea-f75620f3d7dd"}
         },
         {
            "UUID" : {"type" : "FixedString", "value" :
"3c243807-5db3-4172-93a0-0dad00d710f5"}
         },
         {
            "UUID" : {"type" : "FixedString", "value" :
"4bdcaf48-faad-4cd9-befa-19a771205b1d"}
         }
      ]
   }
],
...
```

Now that we have a possible new dialoug option set in place there, we need to add the actual details of that dialog option. It looks like each dialog piece needs a section in the TUT_Start_Brinepool.lsj, so lets add in a section for the uuid we just added.

Quickster\Mods\Quickster\Story\Dialogs\Tutorial\TUT_Start_Brinepool.lsj

```
...
{
    "AllowNodeGrouping" : {"type" : "bool", "value" :
true},
    "GameData" : [
       {
          "AiPersonalities" : [ {} ],
```

```
            "CameraTarget" : {"type" : "int32", "value" :
-1},
            "CustomMovie" : {"type" : "LSString", "value"
: ""},
            "ExtraWaitTime" : {"type" : "int32", "value"
: 0},
            "MusicInstrumentSounds" : [ {} ],
            "OriginSound" : [ {} ],
            "SoundEvent" : {"type" : "LSString", "value"
: ""}
         }
      ],
      "Greeting" : {"type" : "bool", "value" : false},
      "GroupID" : {"type" : "FixedString", "value" : ""},
      "GroupIndex" : {"type" : "int32", "value" : -1},
      "Root" : {"type" : "bool", "value" : false},
      "ShowOnce" : {"type" : "bool", "value" : true},
      "TaggedTexts" : [
         {
            "TaggedText" : [
               {
                  "HasTagRule" : {"type" : "bool",
"value" : true},
                  "RuleGroup" : [
                     {
                        "Rules" : [ {} ],
                        "TagCombineOp" : {"type" :
"uint8", "value" : 0}
                     }
                  ],
                  "TagTexts" : [
                     {
                        "TagText" : [
                           {
                              "LineId" : {"type" :
"guid", "value" : "0ce0944b-6d98-40d5-9737-
17596b0519e6"},
                              "TagText" : {
                                 "handle" :
"h0bdd2c29g6190g4cbdg8cfbg6f4c8478ddce",
                                 "type" :
"TranslatedString",
                                 "version" : 2
                              },
                              "stub" : {"type" : "bool",
"value" : true}
                           }
                        ]
                     }
                  ]
               }
            ]
```

```
      }
   ],
   "Tags" : [ {} ],
   "UUID" : {"type" : "FixedString", "value" :
"3c243807-5db3-4172-93a0-0dad00d710f5"},
   "ValidatedFlags" : [
      {
         "ValidatedHasValue" : {"type" : "bool",
"value" : false}
      }
   ],
   "addressedspeaker" : {"type" : "int32", "value" :
-1},
   "checkflags" : [ {} ],
   "children" : [ {} ],
   "constructor" : {"type" : "FixedString", "value" :
"TagQuestion"},
   "editorData" : [
      {
         "data" : [
            {
               "key" : {"type" : "FixedString",
"value" : "AnimationTags"},
               "val" : {"type" : "LSString", "value" :
""}
            },
            {
               "key" : {"type" : "FixedString",
"value" : "Attitude"},
               "val" : {"type" : "LSString", "value" :
"Default"}
            },
            {
               "key" : {"type" : "FixedString",
"value" : "CinematicNodeContext"},
               "val" : {"type" : "LSString", "value" :
""}
            },
            {
               "key" : {"type" : "FixedString",
"value" : "CinematicObjects"},
               "val" : {"type" : "LSString", "value" :
""}
            },
            {
               "key" : {"type" : "FixedString",
"value" : "CustomCineArtKeysPresent"},
               "val" : {"type" : "LSString", "value" :
"False"}
            },
            {
               "key" : {"type" : "FixedString",
```

```
"value" : "CustomLightingPresent"},
                "val" : {"type" : "LSString", "value" :
"False"}
            },
            {
                "key" : {"type" : "FixedString",
"value" : "CustomSoundPresent"},
                "val" : {"type" : "LSString", "value" :
"False"}
            },
            {
                "key" : {"type" : "FixedString",
"value" : "CustomVFXPresent"},
                "val" : {"type" : "LSString", "value" :
"False"}
            },
            {
                "key" : {"type" : "FixedString",
"value" : "Emotion"},
                "val" : {"type" : "LSString", "value" :
"Default"}
            },
            {
                "key" : {"type" : "FixedString",
"value" : "ForceDisableIsCustomNode"},
                "val" : {"type" : "LSString", "value" :
"False"}
            },
            {
                "key" : {"type" : "FixedString",
"value" : "ID"},
                "val" : {"type" : "LSString", "value" :
"N3"}
            },
            {
                "key" : {"type" : "FixedString",
"value" : "InternalNodeContext"},
                "val" : {"type" : "LSString", "value" :
""}
            },
            {
                "key" : {"type" : "FixedString",
"value" : "IsCustomNode"},
                "val" : {"type" : "LSString", "value" :
"False"}
            },
            {
                "key" : {"type" : "FixedString",
"value" : "MusicTags"},
                "val" : {"type" : "LSString", "value" :
""}
            },
```

```
            {
               "key" : {"type" : "FixedString",
"value" : "NodeContext"},
               "val" : {"type" : "LSString", "value" :
""}
            },
            {
               "key" : {"type" : "FixedString",
"value" : "Quality"},
               "val" : {"type" : "LSString", "value" :
"Bronze"}
            },
            {
               "key" : {"type" : "FixedString",
"value" : "SFXTags"},
               "val" : {"type" : "LSString", "value" :
""}
            },
            {
               "key" : {"type" : "FixedString",
"value" : "StateChangeTags"},
               "val" : {"type" : "LSString", "value" :
""}
            },
            {
               "key" : {"type" : "FixedString",
"value" : "TemplateNodeUUID"},
               "val" : {"type" : "LSString", "value" :
"00000000-0000-0000-0000-000000000000"}
            },
            {
               "key" : {"type" : "FixedString",
"value" : "TemplateVersion"},
               "val" : {"type" : "LSString", "value" :
"0"}
            },
            {
               "key" : {"type" : "FixedString",
"value" : "VFXTags"},
               "val" : {"type" : "LSString", "value" :
""}
            },
            {
               "key" : {"type" : "FixedString",
"value" : "collapsed"},
               "val" : {"type" : "LSString", "value" :
"False"}
            },
            {
               "key" : {"type" : "FixedString",
"value" : "logicalname"},
               "val" : {"type" : "LSString", "value" :
```

```
"Question"}
                },
                {
                   "key" : {"type" : "FixedString",
"value" : "position"},
                   "val" : {"type" : "LSString", "value" :
"15;15"}
                },
                {
                   "key" : {"type" : "FixedString",
"value" : "sourcetemplate"},
                   "val" : {"type" : "LSString", "value" :
""}
                }
            ]
         }
    ],
    "endnode" : {"type" : "bool", "value" : true},
    "exclusive" : {"type" : "bool", "value" : false},
    "gameplaynode" : {"type" : "bool", "value" :
false},
    "optional" : {"type" : "bool", "value" : false},
    "setflags" : [ {} ],
    "speaker" : {"type" : "int32", "value" : 1},
    "stub" : {"type" : "bool", "value" : true},
    "suppresssubtitle" : {"type" : "bool", "value" :
false},
    "transitionmode" : {"type" : "uint8", "value" : 0},
    "waittime" : {"type" : "float", "value" : -1.0}
 },
...
```

Yeah, this thing is bulky. Lets keep it simple. You can see that the uuid I generated has been added to this section (for the uuid, not the part that says line_id, I left that the same). The other thing to note is I generated a handle and added it in. This will be the dialog for our unique class dialog option. I generated h0bdd2c29g6190g4cbdg8cfbg6f4c8478ddce and added in the required localization string. That was a very quick and rushed way to create unique dialog. Lets go do what we actually planned on doing now, making this new dialog we added unique to our class.

Going over all this code is something I will cover later like I said. For now, lets just get a working example of a dialog option being available only for our class. Lets take a closer look at the TaggedTexts entry in our lines we added above, which is where we want to add information about the tag we want associated with that dialog piece. We entered the text that appears in here but now we need to add the tag associated with that text, otherwise anyone can see it. Inside TaggedTexts, we can see `"Rules" : [ {} ],` . Our tag information needs to go into here.

Quickster\Mods\Quickster\Story\Dialogs\Tutorial\TUT_Start_Brinepool.lsj

```
...
"Rules" : [
   {
      "Rule" : [
         {
            "HasChildRules" : {"type" : "bool", "value"
: false},
            "TagCombineOp" : {"type" : "uint8", "value"
: 0},
            "Tags" : [
               {
                  "Tag" : [
                     {
                        "Object" : {"type" : "guid",
"value" : "35add446-b710-4ad1-8dbc-36f99aecc6d5"}
                     }
                  ]
               }
            ],
            "speaker" : {"type" : "int32", "value" : 1}
         }
      ]
   }
],
...
```

Pretty self explanatory, but make sure to include the tag id of your class that you made earlier with that tag file/included in your classdescriptions.lsx for your class. And with this simple change we should now see our unique dialog when we click the brine pool at the beginning. Like I said before, dealing with cutscenes, dialog, timelines, etc can get cumbersome. I only added this entry as it does technically have to do with classes but this was a very quick look at these files just to help you get started with adding in class specific dialog. If you want to trigger certain things from that dialog, well I'm sure you can imagine how complex it may get. Lets just keep moving on to more class specific things, like selectors.

Selectors ⬆️

Selectors are found in our progressions.lsx files. We have already implemented a selector when we added our skills and ability bonuses so you may have a guess as to what they do.They are used for when you need to make some sort of selection menu, as the name applies. For example, if we wanted our character to choose between two different passives, we need to make a selector for it so it will make a menu for us to pick one of the passives on character creation/level up. Inside the selector ()'s, you will need to add the parameters for it. Typically this is the uuid of some list of passives, spells, etc as the first parameter for most functions. THe other parameters can vary between function to function but could be how many options you can choose, the default selections, etc, deliminated by commas. It

would be different for each selector. You saw how adding the selectors for skills and ability bonus added those extra menus onto the screen (if you tried to load your mod up before it, you would have saw it didnt have the menus for them), so lets make a menu for a passive selection by adding another selector. After that, we will use selectors to select a spell list for the class. Lets get started on adding that passive selection menu by creating a new entry in our selection attribute.

Update selector attribute

As I mentioned, there are different selector functions, but we are focusing on SelectPassives(). Like all selectors, we will need to generate a UUID for this as well as fill in the other parameters. I mentioned it in passing above but this UUID is actually the UUID of a list, in this case a list of passives. SelectPassives()'s selectors takes 3 parameters (TODO as far as i know). In this order they are UUID of the list of passives, the amount of options you can select from that list, and the name you use to link to a file we go over later ProgressionDescriptions.lsx . Look at my file below for an example after I added them in.

Quickster\Public\Quickster\Progressions\Progressions.lsx

```
<?xml version="1.0" encoding="utf-8"?>
<save>
  <version major="4" minor="3" revision="0"
build="333"/>
  <region id="Progressions">
    <node id="root">
      <children>
        <node id="Progression">
          <attribute id="Level" type="uint8"
value="1"/>
          <attribute id="Name" type="LSString"
value="Quickster"/>
          <attribute id="ProgressionType" type="uint8"
value="0"/>
          <attribute id="Selectors" type="LSString"
value="SelectPassives(cbbe590a-d630-4fa0-b135-
c16bdf929ad0,1,Quickster_Passives_Level1);SelectSkills(
30dea46e-feae-4637-85ce-
d4cc7a5ae256,2);SelectAbilityBonus(98ef1592-87ad-4b85-
9583-70c3b12037a9,AbilityBonus,2,1)"/>
          <attribute id="TableUUID" type="guid"
value="b283c957-2267-484a-a6c0-f98479c55e53"/>
          <attribute id="UUID" type="guid"
value="cfc536d4-10fa-4342-a447-dd952f4a6df7"/>
        </node>
      </children>
    </node>
  </region>
</save>
```

Now that we have indicated a passive is to be selected at level 1/character creation, lets create the actual file and list associated with that UUID, PassiveLists.lsx

PassiveLists.lsx

If you have been following along, you might have noticed the pattern, at least for selectors. We add a selector, with a uuid. Link that UUID to a list, and from the list we select default options. We did it a bit out of order before for the sake of starting off but lets follow that flow for getting this passive selection for our character, since I think it makes more sense this way and we already updated our Progressions.lsx for it. PassiveLists.lsx, like our other list items, needs a list of strings which refer to our passive options we want to present on the progression. Lets look at attributes we will be changing.

attribute id's

- **Passives** : Will take a list of passives you want your class to be able to select on passive selection on character creation/progression. We will make the actual passive in another file, for now we just need the names to refer to them. I used Quickster_GottaRun and Quickster_LongJump since I have a passive already in mind. You can add more than two or less even.
- **UUID** : A unique UUID for our list. We generated one already which we put in our progressions. I got cbbe590a-d630-4fa0-b135-c16bdf929ad0.

Here is how my file looks after doing all the above:

Quickster\Public\Quickster\Lists\PassiveLists.lsx

```
<?xml version="1.0" encoding="utf-8"?>
<save>
  <version major="4" minor="3" revision="0"
build="333"/>
  <region id="PassiveLists">
    <node id="root">
      <children>
        <node id="PassiveList">
          <attribute id="Passives" type="LSString"
value="Quickster_GottaRun,Quickster_LongJump"/>
          <attribute id="UUID" type="guid"
value="cbbe590a-d630-4fa0-b135-c16bdf929ad0"/>
        </node>
      </children>
    </node>
  </region>
</save>
```

ProgressionDescriptions.lsx

Earlier, I mentioned how SelectPassives last parameter relates to a file called ProgressionDescriptions.lsx. This file will handle linking the description of our

progression (in this case a passive) to a localization entry, so handles like we had for ClassDescriptions.lsx.

attribute id's

- **Description** : The level for this progression. Another way to say what happens to your class when they reach a certain level. Class creation is basically level, we put 1 here.
- **Name** : The name of your class. I put Quickster.
- **DisplayName** : There are 3 different progression types the game will look for. Class Progression = 0, 1 = Subclass Progression, and 2 = Race Progression. We are only making a class for now, so I put 0.
- **SelectorId** : This will link to the string you entered as your third parameter in SelectPassives(). In my case, I named it Quickster_Passives_Level1 since the name makes sense to me for what it is.
- **UUID** : A unique UUID for your progression description. I generated 2f0a696f-92ee-41cc-bbc7-28de32eaa323. As far as I know we need to generate this but we dont need to paste it anywhere else.

Here is how my file looks after doing all the above:

Quickster\Public\Quickster\Progressions\ProgressionsDescriptions.lsx

```
<?xml version="1.0" encoding="utf-8"?>
<save>
  <version major="4" minor="3" revision="0"
build="333"/>
  <region id="ProgressionDescriptions">
    <node id="root">
      <children>
        <node id="ProgressionDescription">
          <attribute id="Description"
type="TranslatedString"
handle="hdb36d078g9bdeg43d6gac66g42018a93c1d1"
version="1"/>
          <attribute id="DisplayName"
type="TranslatedString"
handle="ha63af070gd0b9g4d3dg8cb9g4fbec3b31698"
version="1"/>
          <attribute id="SelectorId" type="FixedString"
value="Quickster_Passives_Level1"/>
          <attribute id="UUID" type="guid"
value="2f0a696f-92ee-41cc-bbc7-28de32eaa323"/>
        </node>
      </children>
    </node>
  </region>
</save>
```

*Did you remember to update your localization file when you made your handles, like I said here?

Passive.txt

As our mod grows, we have to make sure we structure our project/mod folder and its subdirectories correctly. Before we start the next step take a moment to make a few folders. Inside our {mod_name}\Public{mod_name} folder, we need to make a folder called Stats which has a folder called Generated in it. Inside Generated we want to make a folder called Data. And finally, inside that Data folder, we want to make a file called Passive.txt. Your data folder will be used for actions and effects you want characters to use/have. Things like interrupts, passives, spells, etc. You also probably noticed its not a xml/lsx file. This is the first text file we are looking at and its format is different from the lsx as expected. It simply contains entries, a type (in this case PassiveData), and data fields. In our PassiveLists.lsx, we gave our list two passives, Quickster_GottaRun and Quickster_LongJump so I will make an entry for each of them, like this:

```
new entry "Quickster_GottaRun"
type "PassiveData"
```

and

```
new entry "Quickster_LongJump"
type "PassiveData"
```

so now I just need to add the data fields I mentioned earlier. Here is what I did for my first entry.

data name

- **DisplayName** : A handle for what the passive name is.
- **Description** : A handle for what the passive description is.
- **Icon** : String linking to an icon asset. I will use a default one for this, PassiveFeature_FightingStyle_Archery. You can find icon creation here.(TODO)
- **Properties** : I think these are like effects and things like that. We put “Highlighted” but i need to test (TODO)
- **Boosts** : What your passive actually does. To better understand your options, see file insights. I want to make this passive increase my move speed so I found an existing boost that does that and tweaked it to my liking. I used “ActionResourceMultiplier(Movement,175,0)”.

I repeated this for my second entry but tailored to that passive. This meant adding a boost I found that already exists under a entry for “Athlete_StandUp” in the Passive.txt file of the game since it increases jump: data “Boosts”

“JumpMaxDistanceMultiplier(1.5)”. I adjusted my related localizations for DisplayName and Description as well.

Here is how my file looks after doing all the above:

Quickster\Public\Quickster\Stats\Generated\Data\Passive.txt

```
new entry "Quickster_GottaRun"
type "PassiveData"
data "DisplayName"
"hc2cce261g6b48g4d12g8b06g8edacb8f5dea"
data "Description"
"h6f23489eg767ag43e7g9a6agbc0980fa925f"
data "Icon" "PassiveFeature_FightingStyle_Archery"
data "Properties" "Highlighted"
data "Boosts"
"ActionResourceMultiplier(Movement,175,0)"

new entry "Quickster_LongJump"
type "PassiveData"
data "DisplayName"
"h0d1694d6gad2eg4361gb723g70a78e80e615"
data "Description"
"h5dc1bb71g31b4g4c67g9eb0ga6dfcd323191"
data "Icon" "PassiveFeature_FightingStyle_Defense"
data "Properties" "Highlighted"
data "Boosts" "JumpMaxDistanceMultiplier(1.5)"
```

That should do it! Go load up your game and select your class. You should see a new selection menu appear for it where you can select your passive from the two. Once you get control of your character, test out the passive to make sure it works. In my case, I simply turned on turn based mode and could see my character could now move way past the default 10 m thanks to the Gotta Run! passive I added. Lets take a quick look at another type of selector for an example, `SelectSpells` (COMING SOON).

Boosts and PassivesAdded/Removed (Progressions.lsx)⬆️

Our class is really starting to come together. Lets add more to our class to fully flesh it out. This involved adding two/three more attributes to our Progressions.lsx, Boosts and PassivesAdded/Removed.

Boosts

Boosts are for things our class should get, like maybe certain weapon proficiencies, different actions resources (sorcery points, etc), debuffs, buffs, ability changes, etc.

You may remember we also applied boosts earler when we created the passives we selected on creation. Boosts can also be applied via spells/status. If we load up our class right now, we see that we dont have any sort of proficiency in anything (excluding the start proficiencies we get from whatever race you picked)! We suck! No matter, lets get good. Remember our Progressions.lsx file? Lets add in our new attribute. We can add multiple entries at once, just separated by a ;.

Quickster\Public\Quickster\Progressions\Progressions.lsx

```
<?xml version="1.0" encoding="utf-8"?>
<save>
  <version major="4" minor="3" revision="0"
build="333"/>
  <region id="Progressions">
    <node id="root">
      <children>
        <node id="Progression">
          <attribute id="Boosts" type="LSString"
value="ProficiencyBonus(SavingThrow,Dexterity);Proficie
ncyBonus(SavingThrow,Charisma);Proficiency(LightArmor);
Proficiency(SimpleWeapons)"/>
          <attribute id="Level" type="uint8"
value="1"/>
          <attribute id="Name" type="LSString"
value="Quickster"/>
          <attribute id="ProgressionType" type="uint8"
value="0"/>
          <attribute id="Selectors" type="LSString"
value="SelectPassives(cbbe590a-d630-4fa0-b135-
c16bdf929ad0,1,Quickster_Passives_Level1);SelectSkills(
30dea46e-feae-4637-85ce-
d4cc7a5ae256,2);SelectAbilityBonus(98ef1592-87ad-4b85-
9583-
70c3b12037a9,AbilityBonus,2,1);SelectSpells(6a219531-
09df-4216-9656-58f4661cc632,2,0,,,,AlwaysPrepared)"/>
          <attribute id="TableUUID" type="guid"
value="b283c957-2267-484a-a6c0-f98479c55e53"/>
          <attribute id="UUID" type="guid"
value="cfc536d4-10fa-4342-a447-dd952f4a6df7"/>
        </node>
      </children>
    </node>
  </region>
</save>
```

PassivesAdded/Removed

Lets cover two different attributes here that are fairly similar, PassivesAdded and PassivesRemoved. As the name implies these are passives effects our character will be receiving upon levelup (or removing in the case of PassivesRemoved). Okay,

I will admit a bit of confusion on my part here. Boosts to me seems very similar to passives to me so its a bit odd that we dont see these passives being added straight to boosts. It seems like boosts handles things more specific to your class, for example editing basic features for my class like what weapons it can use, what sort of resources will this class use, etc falls into boosts while passives will refer to passives found in Passive.txt, which we cover right after this. You may be wondering when should you use this over putting a SelectPassives() or AddPassives() in a selector and just add the passive that way. I have no idea what is better practice. But to me it makes more sense to only use SelectPassives() in the selector if you need to select an option obviously but if you want to just give something to your class, dont use addpassives() in selectors. Just add it into PassivesAdded attribute. Also I think the bigger reason to use PassivesAdded attribute is when you arent using a passive list but just trying to add a singe passive, like I am in this case. Anyways, lets start by adding in our passive we want our class to get at level 1.

Quickster\Public\Quickster\Progressions\Progressions.lsx

```
<?xml version="1.0" encoding="utf-8"?>
<save>
  <version major="4" minor="3" revision="0"
build="333"/>
  <region id="Progressions">
    <node id="root">
      <children>
        <node id="Progression">
          <attribute id="Boosts" type="LSString"
value="ProficiencyBonus(SavingThrow,Dexterity);Proficie
ncyBonus(SavingThrow,Charisma);Proficiency(LightArmor);
Proficiency(SimpleWeapons)"/>
          <attribute id="Level" type="uint8"
value="1"/>
          <attribute id="Name" type="LSString"
value="Quickster"/>
          <attribute id="PassivesAdded" type="LSString"
value="Quickster_Zoomies"/>
          <attribute id="ProgressionType" type="uint8"
value="0"/>
          <attribute id="Selectors" type="LSString"
value="SelectPassives(cbbe590a-d630-4fa0-b135-
c16bdf929ad0,1,Quickster_Passives_Level1);SelectSkills(
30dea46e-feae-4637-85ce-
d4cc7a5ae256,2);SelectAbilityBonus(98ef1592-87ad-4b85-
9583-
70c3b12037a9,AbilityBonus,2,1);SelectSpells(6a219531-
09df-4216-9656-58f4661cc632,2,0,,,,AlwaysPrepared)"/>
          <attribute id="TableUUID" type="guid"
value="b283c957-2267-484a-a6c0-f98479c55e53"/>
          <attribute id="UUID" type="guid"
value="cfc536d4-10fa-4342-a447-dd952f4a6df7"/>
        </node>
```

```
      </children>
    </node>
  </region>
</save>
```

Here we can see that I added a new attribute, PassivesAdded and I gave it the value Quickster_Zoomies, which I will need to use when making our entry in our Passive.txt file, like we did before for LongJump and GottaRun. Previously since we were creating a passive list that we wanted players to choose from, we had to create some extra files like PassiveLists.lsx. In this case since its just a single passive that will automatically get added to our class, we dont need to add as much abstraction. PassivesRemoved isnt something I have used but I assume it works the same way in reverse. In our example for the quickster mod, I added Quickster_Zoomies, at level 1. But lets say at level 2 we need to replace this passive with a new one, idk maybe like an upgrading passive. This would mean we want to remove Quickster_Zoomies, in which case we would simply use the PassivesRemoved and refer to Quickster_Zoomies. Anyways, lets make this passive a bit more complex. Here is my entry in Passive.txt for Quickster_Zoomies.

Quickster\Public\Quickster\Stats\Generated\Data\Passive.txt

```
new entry "Quickster_Zoomies"
type "PassiveData"
data "DisplayName"
"hcf90a672g10deg4b3egad4fg9094c6ac6224;1"
data "Description"
"he37249cfg5d71g499cg82d9gc369133f2066;2"
data "Icon" "PassiveFeature_ExtraAttack"
data "Properties" "Highlighted"
data "StatsFunctorContext"
"OnCast;OnStatusRemoved;OnStatusApplied"
data "Conditions" "
(context.HasContextFlag(StatsFunctorContext.OnCast) and
ExtraAttackSpellCheck() and HasUseCosts('ActionPoint',
true) and not
Tagged('EXTRA_ATTACK_BLOCKED',context.Source) and not
HasStatus('SLAYER_PLAYER',context.Source) and not
HasStatus('SLAYER_PLAYER_10',context.Source) and
TurnBased(context.Source)) or
(context.HasContextFlag(StatsFunctorContext.OnStatusRem
oved) and StatusId('INITIAL_ATTACK_TECHNICAL') and
TurnBased()) or
(context.HasContextFlag(StatsFunctorContext.OnStatusApp
lied) and StatusId('EXTRA_ATTACK_Q'))"
data "StatsFunctors"
"IF(context.HasContextFlag(StatsFunctorContext.OnCast))
:ApplyStatus(SELF,EXTRA_ATTACK_Q,100,1);IF(context.HasC
ontextFlag(StatsFunctorContext.OnStatusRemoved)):ApplyS
tatus(EXTRA_ATTACK_Q,100,1);IF(context.HasContextFlag(S
tatsFunctorContext.OnStatusApplied) and not
```

```
HasHigherPriorityExtraAttackQueued('EXTRA_ATTACK_Q')
and not HasAnyExtraAttack()):ApplyStatus(EXTRA_ATTACK,
100, 1)"
```

*Originally Quickster_Zoomies was suppose to add more movement hence the name but I eventually changed it to make a second attack instead. Not really important, just if you were confused why a passive that has the name zoomies is meant to give double attacks.

At this point, covering new attribute/data options is some what pointless since my use case wont be the same as yours, assuming you are making a different mod then me. While im not expecting you to know each time some new attribute/data is added, I wont cover each attribute/data in full detail to keep things more clear here. I will give brief explanations on these elements but details explaining what the file, its attributes/data, as well as options for the attributes/data can and should be viewed in file insights page . I will however quickly cover StatFunctorsContext, Conditions, Statfunctors. As you can see, I choose some kinda wacky StatFunctorsContext, Conditions, and Statfunctors. This is just to show you how these items are linked together and how multiple parameters can be sent and then using the conditions and statfunctors we can aim to create specific situation for our passive to go off, especially if we have multiple StatFunctorsContext.

data name

- **StatFunctorsContext** :This data bit is used for what will trigger the passive entry we made (Quickster_Zoomies). I put `OnCast;OnStatusRemoved;OnStatusApplied` . Essentially this is saying we have 3 things that can trigger this, `OnCast` , `OnStatusRemoved` , `OnStatusApplied` . The `;` is being used to separate each statfunctor.
- **Conditions** : Specifies if the the passive should run. If `StatFunctorsContext` is what the passive is waiting for the activate, the condition is like making sure that once our passive triggers, we check if our condition is met which means the passive will work in this case. We have alot of checks, lets look at a bit of the beginning:

```
1 (context.HasContextFlag(StatsFunctorContext.OnCast)
  and ExtraAttackSpellCheck() and
  HasUseCosts('ActionPoint', true) and not ...
```

We can see first we are looking at our first `StatFunctorContext` , `OnCast` . It is checking if our context for this passive being triggered is `OnCast` as well as checking via the function `ExtraAttackSpellCheck()` which most likely checks if we have used a extra attack already (perhaps from multiclassing) and we check if we have an action point, and then we check if something is not but I cut off at that point as I think you get it. You can see grouped calls within parathenses but basically everything is just a chain of and statements.

- **StatFunctors** : What actually happens if our conditions are met. Lets take a look at this somewhat complex entry.

```
IF(context.HasContextFlag(StatsFunctorContext.OnCast
)):ApplyStatus(SELF,EXTRA_ATTACK_Q,100,1);IF(context
.HasContextFlag(StatsFunctorContext.OnStatusRemoved)
):ApplyStatus...
```

  First we check what `StatFunctorContext` parameter was called, so in this case if we were triggered from `OnCast` , we look for the `:` , what is after that is what will happen so in this case we apply a status to ourselves that gives an extra attack. After that we see a `;` which indicated the start of a new statement, ie we are looking at what happens when `OnStatusRemoved` is called.

Like I mentioned, I have added a fairly complex entry for a passive to help show how we can format our entries correctly. Not all of our entries will look this bizarre but this is a good way to demonstrate how we can be triggering off multiple things and then how we process those triggers since we can have something different happen based of the trigger.

USEFUL TESTING STEP ⬆️

While making this class guide, I reached a point where I realized testing my class was kind of difficult. Since we spawn our character on the nautiloid, we need to do a bit of running around to even reach our first enemy. This combined with the fact it takes a long time to load the game as well (for me at least), could lead to alot of time wasted on “debugging” when you arent even doing any debugging, just getting to a location in game where you can debug/test your class. Maybe in the future we can create a special debug area our character spawns in for initial testing but now my work around is to give our class the ability to spawn a training dummy to test out moves on. Im not going to cover what I do here right now since im just adding this portion in here as a quick setup step so you aren’t confused when my class randomly summons a mob to fight. TODO add a link to how to create the summon.

Maybe you noticed in the previous section when I was going over PassivesAdded and Boosts, my progressions file had a new entry in the selectors, SelectSpells. I plan on covering spells later (TODO link spells section) so dont worry about it for now. All you need to know is that I added in the required stuff to summon the rangers familiars. Like I said above, I did this so I have something to test my spells/attacks/passives/etc on right away when the game starts.

Subclasses ⬆️

Here we are, the final section of this basic class creation guide. The reason I put subclasses last is because they kind of follow the same rules as your actual class. For example, we will need to make a new class description entry, a new progression

entry, etc. These will have their own tags and own spells, just like a class would. The only realy difference is that it will be considered a child to the base class (or the base class is the parent, whatever lingo works better for you). So basically, if you understand how to make a class, you probably know how to make a subclass as well but just dont know it yet. Im going to make two subclasses that can be selected on character creation.

Make a Subclass

Lets make our way back to the ClassDescriptions.lsx, where we first entered our Quickster class. Here it is again for reference since its been a minute.

Quickster\Public\Quickster\ClassDescription\ClassDescription.lsx

```
<?xml version="1.0" encoding="utf-8"?>
<save>
  <version major="4" minor="3" revision="0"
build="333"/>
  <region id="ClassDescriptions">
    <node id="root">
      <children>
        <node id="ClassDescription">
          <attribute id="BaseHp" type="int32"
value="6"/>
          <attribute id="CharacterCreationPose"
type="guid" value="0f07ec6e-4ef0-434e-9a51-
1353260ccff8"/>
          <attribute id="Description"
type="TranslatedString"
handle="hc25377a0g8e44g4645g90c5gee05d6c5e31b"
version="1"/>
          <attribute id="DisplayName"
type="TranslatedString"
handle="heb6d4970g5238g4bb8ga932g9dd4357d61ed"
version="1"/>
          <attribute id="HpPerLevel" type="int32"
value="4"/>
          <attribute id="LearningStrategy" type="uint8"
value="1"/>
          <attribute id="Name" type="FixedString"
value="Quickster"/>
          <attribute id="PrimaryAbility" type="uint8"
value="6"/>
          <attribute id="ProgressionTableUUID"
type="guid" value="b283c957-2267-484a-a6c0-
f98479c55e53"/>
          <attribute id="SoundClassType"
type="FixedString" value="Paladin"/>
          <attribute id="SpellCastingAbility"
type="uint8" value="2"/>
          <attribute id="UUID" type="guid"
```

```
value="e7b0f304-da32-410e-9e58-6efea9272673"/>
          <children>
              <node id="Tags">
                  <attribute id="Object" type="guid"
value="35add446-b710-4ad1-8dbc-36f99aecc6d5"/>
              </node>
          </children>
        </node>
      </children>
    </node>
  </region>
</save>
```

Subclasses follow the similar rules as a normal class when we define them in ClassDescriptions. They get their own node for ClassDescription. Lets take a look, it will look quite similar to above. Lets add two subclasses for our class that we can choose at character creation.
Quickster\Public\Quickster\ClassDescription\ClassDescription.lsx

```
<?xml version="1.0" encoding="utf-8"?>
<save>
  <version major="4" minor="3" revision="0"
build="333"/>
  <region id="ClassDescriptions">
    <node id="root">
      <children>
        <node id="ClassDescription">
          <attribute id="BaseHp" type="int32"
value="6"/>
          <attribute id="CharacterCreationPose"
type="guid" value="0f07ec6e-4ef0-434e-9a51-
1353260ccff8"/>
          <attribute id="Description"
type="TranslatedString"
handle="hc25377a0g8e44g4645g90c5gee05d6c5e31b"
version="1"/>
          <attribute id="DisplayName"
type="TranslatedString"
handle="heb6d4970g5238g4bb8ga932g9dd4357d61ed"
version="1"/>
          <attribute id="HpPerLevel" type="int32"
value="4"/>
          <attribute id="LearningStrategy" type="uint8"
value="1"/>
          <attribute id="Name" type="FixedString"
value="Quickster"/>
          <attribute id="PrimaryAbility" type="uint8"
value="6"/>
          <attribute id="ProgressionTableUUID"
type="guid" value="b283c957-2267-484a-a6c0-
```

```
f98479c55e53"/>
          <attribute id="SoundClassType"
type="FixedString" value="Paladin"/>
          <attribute id="SpellCastingAbility"
type="uint8" value="2"/>
          <attribute id="UUID" type="guid"
value="e7b0f304-da32-410e-9e58-6efea9272673"/>
          <children>
              <node id="Tags">
                  <attribute id="Object" type="guid"
value="35add446-b710-4ad1-8dbc-36f99aecc6d5"/>
              </node>
          </children>
        </node>
        <node id="ClassDescription">
          <attribute id="CharacterCreationPose"
type="guid" value="0f07ec6e-4ef0-434e-9a51-
1353260ccff8" />
          <attribute id="Description"
type="TranslatedString"
handle="hffe64678g80cbg4c7fgba84g4fc31f2d99e4"
version="1" />
          <attribute id="DisplayName"
type="TranslatedString"
handle="h9a8d0d48ga969g481agb402g8ed1b0736135"
version="1" />
          <attribute id="LearningStrategy" type="uint8"
value="1" />
          <attribute id="Name" type="FixedString"
value="Magical Quickster" />
          <attribute id="ParentGuid" type="guid"
value="e7b0f304-da32-410e-9e58-6efea9272673" />
          <attribute id="PrimaryAbility" type="uint8"
value="6" />
          <attribute id="ProgressionTableUUID"
type="guid" value="5c55e2a8-5615-4312-bca3-
de0ae5e0190e" />
          <attribute id="ShortName"
type="TranslatedString"
handle="hf587f6c9geac5g4198g9b00gd6af22b057f0"
version="1" />
          <attribute id="SoundClassType"
type="FixedString" value="Paladin" />
          <attribute id="SpellCastingAbility"
type="uint8" value="6" />
          <attribute id="UUID" type="guid"
value="6c824863-0bb3-48ac-8a63-0b3915451069" />
        </node>
        <node id="ClassDescription">
          <attribute id="CharacterCreationPose"
type="guid" value="0f07ec6e-4ef0-434e-9a51-
1353260ccff8" />
```

```
            <attribute id="Description"
type="TranslatedString"
handle="h65d4315bg1722g440cg9fc0gf7f636f7de8f"
version="1" />
            <attribute id="DisplayName"
type="TranslatedString"
handle="h4acf2600ge9f6g4d2egafa4g83de78cec3b5"
version="1" />
            <attribute id="LearningStrategy" type="uint8"
value="1" />
            <attribute id="Name" type="FixedString"
value="Physical Quickster" />
            <attribute id="ParentGuid" type="guid"
value="e7b0f304-da32-410e-9e58-6efea9272673" />
            <attribute id="PrimaryAbility" type="uint8"
value="2" />
            <attribute id="ProgressionTableUUID"
type="guid" value="944e2b7b-0145-404d-b3df-
3df8d5ae4d01" />
            <attribute id="ShortName"
type="TranslatedString"
handle="h9aa8f380ge730g4b6cga22agbc07dfb7fcd6"
version="1" />
            <attribute id="SoundClassType"
type="FixedString" value="Paladin" />
            <attribute id="SpellCastingAbility"
type="uint8" value="2" />
            <attribute id="UUID" type="guid"
value="ff3bf3e7-12ee-4183-8ac6-a9aefb0898f4" />
          </node>
        </children>
      </node>
    </region>
</save>
```

As you can see, most of the information looks the same. The only key values you need to worry about here are as followed:

- **ParentGuid** : This should be the uuid of the parent class, so in this case I grabbed the UUID from my Quickster ClassDescription UUID and applied it to both classes.
- **ProgressionTableUUID** : We need to generate a new uuid for this. We will be making a new progression for our subclass, it should not be the same uuid as your main classes progression table and it should also have a different UUID then the other subclass.

Make sure you generate UUID's and handles where needed and fix your Name as well to your subclass. Any changes we make on our subclass will be put ontop of our base class. Now lets make a progressions entry for our subclass, similar to how we made out progressions entry for our class.

Since im not to concerned about fleshing out my subclasses (since this is a tutorial and alot of what I would cover in making a subclass is essentially the same as making a class), I only intend to make a simple progression to show how a subclass progression entry is formatted. Much of what is entered is the same as a normal class progression as you will see. Here is my new progressions.lsx file after adding in subclass progressions for my main class.

Quickster\Public\Quickster\Progressions\Progressions.lsx

```
<?xml version="1.0" encoding="utf-8"?>
<save>
  <version major="4" minor="3" revision="0"
build="333"/>
  <region id="Progressions">
    <node id="root">
      <children>
        <node id="Progression">
          <attribute id="Boosts" type="LSString"
value="ProficiencyBonus(SavingThrow,Dexterity);Proficie
ncyBonus(SavingThrow,Charisma);Proficiency(LightArmor);
Proficiency(SimpleWeapons)"/>
          <attribute id="Level" type="uint8"
value="1"/>
          <attribute id="Name" type="LSString"
value="Quickster"/>
          <attribute id="PassivesAdded" type="LSString"
value="Quickster_Zoomies"/>
          <attribute id="ProgressionType" type="uint8"
value="0"/>
          <attribute id="Selectors" type="LSString"
value="SelectPassives(cbbe590a-d630-4fa0-b135-
c16bdf929ad0,1,Quickster_Passives_Level1);SelectSkills(
30dea46e-feae-4637-85ce-
d4cc7a5ae256,2);SelectAbilityBonus(98ef1592-87ad-4b85-
9583-
70c3b12037a9,AbilityBonus,2,1);SelectSpells(6a219531-
09df-4216-9656-58f4661cc632,2,0,,,,AlwaysPrepared)"/>
          <attribute id="TableUUID" type="guid"
value="b283c957-2267-484a-a6c0-f98479c55e53"/>
          <attribute id="UUID" type="guid"
value="cfc536d4-10fa-4342-a447-dd952f4a6df7"/>
          <children>
            <node id="SubClasses">
              <children>
                <node id="SubClass">
                  <attribute id="Object" type="guid"
value="6c824863-0bb3-48ac-8a63-0b3915451069" />
                </node>
                <node id="SubClass">
                  <attribute id="Object" type="guid"
value="ff3bf3e7-12ee-4183-8ac6-a9aefb0898f4" />
```

```
                </node>
              </children>
            </node>
          </children>
        </node>
        <node id="Progression">
          <attribute id="Boosts" type="LSString"
value="ActionResource(SpellSlot,2,1)"/>
          <attribute id="Level" type="uint8"
value="1"/>
          <attribute id="Name" type="LSString"
value="Magical Quickster"/>
          <attribute id="PassivesAdded" type="LSString"
value=""/>
          <attribute id="ProgressionType" type="uint8"
value="1"/>
          <attribute id="Selectors" type="LSString"
value=""/>
          <attribute id="TableUUID" type="guid"
value="5c55e2a8-5615-4312-bca3-de0ae5e0190e"/>
          <attribute id="UUID" type="guid"
value="4b9c5267-5512-41f3-84cd-5b3358df9eaa"/>
        </node>
        <node id="Progression">
          <attribute id="Boosts" type="LSString"
value=""/>
          <attribute id="Level" type="uint8"
value="1"/>
          <attribute id="Name" type="LSString"
value="Physical Quickster"/>
          <attribute id="PassivesAdded" type="LSString"
value=""/>
          <attribute id="ProgressionType" type="uint8"
value="1"/>
          <attribute id="Selectors" type="LSString"
value=""/>
          <attribute id="TableUUID" type="guid"
value="944e2b7b-0145-404d-b3df-3df8d5ae4d01"/>
          <attribute id="UUID" type="guid"
value="3764c46a-94e3-47b4-b6b0-bc22304ba686"/>
        </node>
      </children>
    </node>
  </region>
</save>
```

As you can see, our subclasses progressions look alot like our classes! Lets go over what was added and why:

- **Our new children node with the sub node id=SubClasses** : This is the only thing we have to change in regard to our original progression entry for our main class. We make these sub nodes to declare what subclasses are given at that progression, noted by the UUID of the subclass, which is the UUID we made for it in our ClassDescriptions.lsx file for our subclasses.
- **ProgressionType** : Both of our subclasses need to have this a 1. I mentioned it before but 0 is for classes, 1 is for subclasses, 2 is for races.
- **TableUUID** : Use the ProgressionTableUUID we made in ClassDescriptions.lsx. Remember, we made one for each subclass, make sure your dont mix them up when applying them here.

I didnt really give any of the subclasses anything since, like I said a million times, its pretty much the same as adding stuff for your main class progression. I did add `<attribute id="Boosts" type="LSString" value="ActionResource(SpellSlot,2,1)"/>` in my Magical Quickster Subclass progression just to show it. If I load up the game as our quickster class and select the Magic Quickster subclass, once I get in game I will now have 2 spell slots since I specified the subclass gets it. My other subclass which doesnt have that, will not receive the spellslots.

Action Resources ⬆️

Okay I intended to stop after subclasses but there is one more optional thing to consider. Something I should have covered before (sorry)! That would be Actions and more specifically, the ActionResourceDefinitions.lsx file. When you think of actions, im sure you think about the base action point you get every turn, or maybe the bonus action you get. Both of these are actions but we also know about some other actions, like spell slots, rage, sorcery points, etc. Basically, it is a resource that our class uses, typically with a certain number of charges, ways to restore them, special effects, etc.

Create an Action Resource

We should start by making a new folder and file inside it. Inside the Public/{Mod Name}/ (so for me Public/Quickster/), you will want to make a folder called ActionResourceDefinitions and inside that make a file called ActionResourceDefinitions.lsx. Lets take a look at an entry I made for my class and go over it.

Quickster\Public\Quickster\ActionResourceDefinitions\ActionResourceDefinitions.lsx

```
<?xml version="1.0" encoding="UTF-8"?>
<save>
    <version major="4" minor="3" revision="0"
build="333"/>
    <region id="ActionResourceDefinitions">
        <node id="root">

```

```
            <children>
                <node id="ActionResourceDefinition">
                    <attribute id="Description"
type="TranslatedString"
handle="hb2d26394gdc0bg458egafa8g7906e8441c87"
version="1"/>
                    <attribute id="DisplayName"
type="TranslatedString"
handle="hf89fcccfg40b1g4671g9e8ege9c0e4e93a8d"
version="1"/>
                    <attribute id="MaxLevel"
type="uint32" value="0"/>
                    <attribute id="Name"
type="FixedString" value="SpeedForce"/>
                    <attribute id="ReplenishType"
type="FixedString" value="ShortRest"/>
                    <attribute id="IsSpellResource"
type="bool" value="false"/>
                    <attribute
id="ShowOnActionResourcePanel" type="bool"
value="true"/>
                    <attribute id="UUID" type="guid"
value="dc2d6ab6-4c86-4f2a-9455-ec86d53a8175"/>
                </node>
            </children>
        </node>
    </region>
</save>
```

We have made a new action resource but right now it just floating around. Like everything we have done, we need to link it to our class, more specifially our progression for our class. Lets go back to our progressions.lsx and take a look at our main class progression since I want it to be applied to the class as a whole and not a specific subclass like I did with my spell slots. Here is my progressions after I attach my newly made action resource to my class.

Quickster\Public\Quickster\Progressions\Progressions.lsx

```
...
 <node id="Progression">
  <attribute id="Boosts" type="LSString"
value="ActionResource(SpeedForce,2,0);ProficiencyBonus(
SavingThrow,Dexterity);ProficiencyBonus(SavingThrow,Cha
risma);Proficiency(LightArmor);Proficiency(SimpleWeapon
s)"/>
  <attribute id="Level" type="uint8" value="1"/>
  <attribute id="Name" type="LSString"
value="Quickster"/>
  <attribute id="PassivesAdded" type="LSString"
value="Quickster_Zoomies"/>
```

```
  <attribute id="ProgressionType" type="uint8"
value="0"/>
  <attribute id="Selectors" type="LSString"
value="SelectPassives(cbbe590a-d630-4fa0-b135-
c16bdf929ad0,1,Quickster_Passives_Level1);SelectSkills(
30dea46e-feae-4637-85ce-
d4cc7a5ae256,2);SelectAbilityBonus(98ef1592-87ad-4b85-
9583-
70c3b12037a9,AbilityBonus,2,1);SelectSpells(6a219531-
09df-4216-9656-58f4661cc632,2,0,,,,AlwaysPrepared)"/>
  <attribute id="TableUUID" type="guid"
value="b283c957-2267-484a-a6c0-f98479c55e53"/>
  <attribute id="UUID" type="guid" value="cfc536d4-
10fa-4342-a447-dd952f4a6df7"/>
  <children>
    <node id="SubClasses">
      <children>
        <node id="SubClass">
          <attribute id="Object" type="guid"
value="6c824863-0bb3-48ac-8a63-0b3915451069" />
        </node>
        <node id="SubClass">
          <attribute id="Object" type="guid"
value="ff3bf3e7-12ee-4183-8ac6-a9aefb0898f4" />
        </node>
      </children>
    </node>
  </children>
</node>
...
```

The only new thing I added was in Boosts. I added the Function ActionResource(). The first parameter is the name of my action resource, so SpeedForce. The second parameter is an int for how many of the resource you get. I want two charges, so I put 2. The last parameter is an int for the level. In my case I dont have a level restriction so I put 0. Thats pretty much all it takes to make an action resource. It may not mean much to you right now but when you make a spell you can refer to this action resource for its use costs.

Starting Equipment ⬆️

Running around Faerun naked is not advised. Most classes would want starting armor, weapon, and objects. Lets get started with giving a class a weapon as well as some armor and objects, which gets defined in the file Equipment.txt. Equipment.txt references items found in files inside the Shared\Stats\Generated\Data folder so lets start with that. I will be giving the class existing ingame items. I wont cover creating new items here for this class so dont expect that below. Please look in the wiki for item creation guides for that.

Data Folder Files

Before we even take a look at the games equipment file, you should get an idea of what items your class will need. Everything your class will reference in its equipment entry can be found in the Data folder as I mentioned above. Specifically, the ones we are concerned about are Weapon.txt, Armor.txt, and Object.txt, at least in the base game. Unless im mistaken, all files in the Data folder could be merged into one file or mixed around. The base game has them seperated for clarity I believe. Its just easier to view it seperated like this but as long as your entry for something in the Data folder has the correct type (ex. `type "Armor"`) to demonstrate what type that entry is, it could all be the same file, mixed, etc and given whatever name(s) you see fit (i think). Data types for weapons, armors, objects arent to crazy unless you want to get wild with boosts so lets just look at some entries for the three that will be added to quickster. (Remember that since these entries exists in the base game, its not like we need to make these data entries. Im only including them below for reference) Data files go far beyond just items though, so check the wiki if you want more information on them.

Weapon.txt

Lets take a look at an entry I plan to give the class.

```
new entry "WPN_Dagger"
type "Weapon"
using "_BaseWeapon"
data "RootTemplate" "569b0f3d-abcd-4b01-aaf0-
979091288163"
data "Damage Type" "Piercing"
data "Damage" "1d4"
data "ValueScale" "0.5"
data "Weight" "0.45"
data "BoostsOnEquipMainHand"
"UnlockSpell(Target_PiercingThrust)"
data "Weapon Group" "SimpleMeleeWeapon"
data "Weapon Properties"
"Finesse;Light;Thrown;Melee;Dippable"
data "Proficiency Group" "Daggers;SimpleWeapons"

new entry "WPN_HandCrossbow"
type "Weapon"
using "_BaseWeapon"
data "RootTemplate" "a5d843ab-c3af-4e60-a925-
bb2e15828938"
data "Damage Type" "Piercing"
data "Damage" "1d6"
data "Damage Range" "3000"
data "WeaponRange" "1500"
data "ValueLevel" "3"
data "Weight" "0.9"
data "Slot" "Ranged Main Weapon"
data "Projectile" "ff93ba9c-124c-454e-ac8c-
```

```
436c136bcef2"
data "BoostsOnEquipMainHand"
"UnlockSpell(Projectile_PiercingShot);UnlockSpell(Proje
ctile_MobileShooting)"
data "Weapon Group" "MartialRangedWeapon"
data "Weapon Properties"
"Ammunition;Loading;Light;Dippable"
data "Proficiency Group" "HandCrossbows;MartialWeapons"
```

Armor.txt

Lets take a look at some entries I plan to give the class.

```
new entry "ARM_Boots_Leather"
type "Armor"
using "_Foot"
data "RootTemplate" "969bab00-b269-46a5-a7e9-
dd4887814719"
data "ValueLevel" "1"
data "ArmorType" "Leather"

new entry "ARM_Robe_Body"
type "Armor"
using "_Body"
data "RootTemplate" "168b9099-19f5-44e4-b55c-
e64ceb60b71f"
data "ArmorClass" "10"
data "ValueUUID" "4c5217d8-0232-4592-9e32-2fd729123f53"
data "ValueScale" "0.8"
data "Weight" "1.8"
data "MinAmount" "1"
data "MaxAmount" "1"
data "Priority" "1"
data "MinLevel" "1"
data "Armor Class Ability" "Dexterity"
```

*Not bothering to add camp clothes entires here but make sure you place those below the armor you want equipped. Order matters I believe.

Object.txt

I think at this point you get the idea. Im not going to worry about adding any new items but I will make sure that my class gets the initial scroll of revivify, keychain, alchemy pouch, and camp supplies backpack, which you will see when I create my actual Equipment.txt file below.

Equipment.txt

The game defines all starting gear for all entities in a file called Equipment.txt, which is found in the \Shared\Stats\Generated folder. Lets take a look at an entry or

two.

```
new equipment "EQP_CC_Barbarian"
add initialweaponset "Melee"
add equipmentgroup
add equipment entry "WPN_Greataxe"
add equipmentgroup
add equipment entry "OBJ_Potion_Healing"
add equipmentgroup
add equipment entry "OBJ_Potion_Healing"
add equipmentgroup
add equipment entry "OBJ_Scroll_Revivify"
add equipmentgroup
add equipment entry "ARM_Shoes_Barbarian"
add equipmentgroup
add equipment entry "ARM_Barbarian"
add equipmentgroup
add equipment entry "WPN_Handaxe"
add equipmentgroup
add equipment entry "WPN_Handaxe"
add equipmentgroup
add equipment entry "OBJ_Keychain"
add equipmentgroup
add equipment entry "OBJ_Bag_AlchemyPouch"
add equipmentgroup
add equipment entry "ARM_Camp_Body"
add equipmentgroup
add equipment entry "ARM_Camp_Shoes"
add equipmentgroup
add equipment entry "OBJ_Backpack_CampSupplies"

new equipment "EQP_CC_Bard"
add initialweaponset "Melee"
add equipmentgroup
add equipment entry "WPN_Rapier"
add equipmentgroup
add equipment entry "WPN_HandCrossbow"
add equipmentgroup
add equipment entry "OBJ_Potion_Healing"
add equipmentgroup
add equipment entry "OBJ_Potion_Healing"
add equipmentgroup
add equipment entry "ARM_Bard"
add equipmentgroup
add equipment entry "ARM_Boots_Leather"
add equipmentgroup
add equipment entry "OBJ_Scroll_Revivify"
add equipmentgroup
add equipment entry "OBJ_Keychain"
add equipmentgroup
add equipment entry "OBJ_Bag_AlchemyPouch"

```

```
add equipmentgroup
add equipment entry "ARM_Camp_Body"
add equipmentgroup
add equipment entry "ARM_Camp_Shoes"
add equipmentgroup
add equipment entry "OBJ_Backpack_CampSupplies"
```

Here we can see entries for the Barbarian and and Bard respectively, as we can tell from their entry name. You can see the names of things in here are somewhat similar to the items we looks through earlier that we wanted to add to our class. Lets make a Equipment.txt file and add in a equipment entry for our class.

Quickster\Public\Quickster\Stats\Generated\Equipment.txt

```
new equipment "EQP_CC_Quickster"
add initialweaponset "Melee"
add equipmentgroup
add equipment entry "WPN_Dagger"
add equipmentgroup
add equipment entry "WPN_HandCrossbow"
add equipmentgroup
add equipment entry "OBJ_Potion_Healing"
add equipmentgroup
add equipment entry "OBJ_Potion_Healing"
add equipmentgroup
add equipment entry "ARM_Robe_Body"
add equipmentgroup
add equipment entry "ARM_Boots_Leather"
add equipmentgroup
add equipment entry "OBJ_Scroll_Revivify"
add equipmentgroup
add equipment entry "OBJ_Keychain"
add equipmentgroup
add equipment entry "OBJ_Bag_AlchemyPouch"
add equipmentgroup
add equipment entry "ARM_Camp_Body"
add equipmentgroup
add equipment entry "ARM_Camp_Shoes"
add equipmentgroup
add equipment entry "OBJ_Backpack_CampSupplies"
```

Nothing unexpected here I think. We have one more thing we need to do, link this equipment entry back to our class. To do that we will look at a file we havent touched in a while, ClassDescriptions.lsx.

##ClassDescriptions.lsx
We only need to make a minor addition here, just one line. To assign a starting equipment to your class, you need to add the attribute `ClassEquipment` and make its value the same as the name of your equipment entry. So for quickster that

would be EQP_CC_Quickster. Take a look.
Quickster\Public\Quickster\ClassDescription\ClassDescription.lsx

```
...
<node id="ClassDescription">
  <attribute id="BaseHp" type="int32" value="6"/>
  <attribute id="CharacterCreationPose" type="guid"
value="0f07ec6e-4ef0-434e-9a51-1353260ccff8"/>
  <attribute id="ClassEquipment" type="FixedString"
value="EQP_CC_Quickster"/>
  <attribute id="Description" type="TranslatedString"
handle="hc25377a0g8e44g4645g90c5gee05d6c5e31b"
version="1"/>
  <attribute id="DisplayName" type="TranslatedString"
handle="heb6d4970g5238g4bb8ga932g9dd4357d61ed"
version="1"/>
  <attribute id="HpPerLevel" type="int32" value="4"/>
  <attribute id="LearningStrategy" type="uint8"
value="1"/>
  <attribute id="Name" type="FixedString"
value="Quickster"/>
  <attribute id="PrimaryAbility" type="uint8"
value="6"/>
  <attribute id="ProgressionTableUUID" type="guid"
value="b283c957-2267-484a-a6c0-f98479c55e53"/>
  <attribute id="SoundClassType" type="FixedString"
value="Paladin"/>
  <attribute id="SpellCastingAbility" type="uint8"
value="2"/>
  <attribute id="UUID" type="guid" value="e7b0f304-
da32-410e-9e58-6efea9272673"/>
  <children>
    <node id="Tags">
      <attribute id="Object" type="guid"
value="35add446-b710-4ad1-8dbc-36f99aecc6d5"/>
    </node>
  </children>
</node>
...
```

This should be all you need for now. If you load up your class now, you should see that your class spawns with the specified equipment.

I think thats it for classes for now. If something class related is missing here or incorrect, please leave a comment.

Return to top